Goldsmiths
UNIVERSITY OF LONDON

An Innovative Internet Relay Chat Application

Author:
Zhao Wang

Supervisor:
Lahcen Ouarbya

BSc Computer Science

2024

Declaration

This project report is submitted in partial fulfilment of the requirements for the degree of BSc Computer Science. I declare that this thesis was composed by myself, that the work contained therein is my own, except where explicitly stated otherwise in the text, and that it has not been submitted, in whole or in part, for any other degree or professional qualification.

I acknowledge the use of ChatGPT (https://chat.openai.com/) to refine the academic language of my own work.
On 20 April 2024 I entered the following prompts:

- synonyms for "front-end web frameworks"
- synonyms for "web views"
- synonym for "achieve"
- is this correct: "to opt-in this new feature"

The output from the generative artificial intelligence was used as a guide for better language and vocabulary, but no generated text was included in the submitted essay.

Zhao Wang

Word Count: 8290 (excluding the index, the declaration, references and appendixes)
This thesis was conducted under the supervision of Lahcen Ouarbya.

Abstract

IRC was introduced in the 90s and documented by Internet Engineering Task Force (IETF) as Request for Comments (RFC) 1459. Although decades have passed since then, IRC still lacks a user-friendly UI. This study explores a highly-scalable implementation.

Keywords

Internet Relay Chat, Django Web Framework, Phoenix Web Framework, Web Development

Contents

Declaration i

Abstract ii

Keywords iii

1 Introduction 1

1.1 Motivation 1

1.2 Problem 2

1.3 Solution 2

1.4 Notable Aspects 2

1.4.1 Rapid Prototyping 2

1.4.2 Open Telecom Platform 2

1.4.3 WebSocket-Native User Interface 3

2 Background 4

2.1 Research 4

2.2 Competition 5

2.3 Other Inspiration 6

3 Methods & Libraries 8

3.1 Method 8

3.2 Libraries 8

3.3 Aims & Objectives 9

3.4 Requirements 9

3.4.1 Requirement Specification 9

3.4.2 Functional Specification 10
3.5 System Design 13
3.5.1 Use Cases 13
3.5.2 Pseudocode 14
3.5.3 UI Wireframes 16
3.5.4 Database 18
3.5.5 System 19
3.6 Development Process 20
3.6.1 Core Technologies 20
3.6.2 Development 21
3.6.3 Testing 22
3.6.4 Evaluation 22
3.7 Timeline 22
3.8 Risks and Risk Management 23
3.8.1 Unfamiliar Technology 23
3.8.2 Project Complexity 24
3.9 Contingency Plan: Reuse the Prototype 24

4 Results: Prototype with Django Web Framework 25
4.1 Code Repository 25
4.2 Core Aspect 25
4.3 Core Technology 26
4.3.1 Web Development 26
4.3.2 Parallel Programming 26
4.3.3 Network Programming 26
4.4 Features and Tests 26
4.4.1 Establishing IRC Connections 26
4.4.2 Basic Messaging Endpoints & UI 27
4.4.3 Basic Client Status Endpoints & UI 27
4.5 Additional Technology 28
4.5.1 Caching 28

4.5.2 Message Queues . . . 28
4.5.3 Phoenix LiveView . . . 28
4.5.4 IRCv3 . . . 29
4.6 Demonstration . . . 29
4.6.1 UI . . . 29
4.6.2 Database . . . 30
4.6.3 Sample Code . . . 31
4.6.4 Tests . . . 32
4.7 Additional Ethical Considerations . . . 33
4.7.1 GDPR . . . 33
4.7.2 The Path to GDPR . . . 33

5 Results: Refactored Implementation 34

5.1 Code Repository . . . 34
5.2 Connection Map . . . 35
5.3 Utility Functions . . . 35
5.4 Configuration Dashboard . . . 35
5.5 IRC Channel Hub . . . 35
5.6 Direct Messaging Page . . . 35
5.7 Implementation Details . . . 36
5.7.1 Connecting to the Database (PostgreSQL) . . . 36
5.7.2 User Registration & Login . . . 37
5.7.3 Group Chat . . . 38
5.7.4 Private Chat . . . 39
5.8 Chat Clients . . . 39
5.9 Utilities . . . 40
5.9.1 *Nix* Package Manager . . . 40
5.9.2 *Docker* Containerization Toolkit . . . 40

6 Discussion 42

6.1 Testing & Evaluation . . . 42
6.1.1 End-to-End Testing . . . 42

6.1.2 Unit Tests 44
6.1.3 Evaluation 45
6.2 Technical Details & Technologies 48
6.2.1 Novel Approaches 48
6.2.2 ExIRC: An IRC Library for Elixir 48
6.3 Challenges & Difficulties 49
6.3.1 Different Programming Paradigms 49
6.3.2 Incomplete Tool-Chain 50

7 Limitations and Future Works 51
7.1 Next Steps 51
7.1.1 Better Documentation & Tutorials 51
7.1.2 Native Mobile Apps 51
7.2 Further Development 51
7.2.1 Better User Experience (UX) 51
7.2.2 HTTP API 52
7.2.3 Embeddable Chatbox Widget and Other Integratable Components . . . 52
7.2.4 Native Support for XMPP & Other Open Messsaing Standards 53
7.2.5 High Availability 53

8 Conclusion 54

References 55

A Appendix: Erlang Build Errors 65

B Appendix: Create User Accounts in iex REPL 66

C Appendix: Database Migration Output 68

D Appendix: ExIRC Connect to Server 70

E Appendix: *Mix test* Results 71

Chapter 1

Introduction

This thesis is about designing and implementing a chat platform using the Internet Relay Chat (IRC) [1] protocol, to welcome new users to IRC [1] networks.

> Sharing knowledge is the most fundamental act of friendship. Because it is a way you can give something without losing something. - Richard M. Stallman

This study is for a reference implementation of a full-stack solution, similar to Weston as to Wayland [2]. The focus here is also "correctness, reliability, predictability, and performance" [3].

1.1 Motivation

Nowadays, many seasoned programmers and open-source software communities are still using IRC [1] as their most important means of communication. However, many people have adopted other messaging applications such as Discord and Slack, which has rendered IRC [1] almost obsolete. As the IRCv3 [4] specifications are being rolled out, many essential features such as chat history [5] are standardized, much more elegantly than its proprietary competitors, and it is time to create an IRC [1] web client with these new standards.

1.2 Problem

There are some self-hosted webchat solutions for the IRC [1] protocol, such as The Lounge [6] and Convos [7], with modern web interfaces, but it is quite impossible for a non-technical user to set up such services. As for managed set-up, IRCCloud [8] is a freemium SaaS [9] solution with a minimal and intuitive design.

Only with open-source utilities, it is difficult to get an IRC [1] client up and running properly, and it is even harder to register an IRC nickname [10] on a cultured IRC [1] network, since most of reputable IRC [1] networks use different kinds of verification and anti-spam mechanisms to keep a relatively high quality user base. This has made many people turn to more beginner-friendly but less user-centric software such as Discord for collaboration.

1.3 Solution

This project aims to get potential users started, with social login [11] options, such as Google OAuth [12] and GitHub OAuth [13], built into the web interface. For ease of use, automated nickname [10] registration will be enabled with IRC [1] networks' consent. High quality IRC [1] networks, such as Libera.Chat [14], will also be recommended, to make sure that the users have a great start.

1.4 Notable Aspects

1.4.1 Rapid Prototyping

Although Django [15] and Python [16] are known not to be optimal for this project, Django REST framework [17] was used to implement the minimal viable product (MVP), for its capability of quick prototyping.

1.4.2 Open Telecom Platform

The production implementation uses Open Telecom Platform (OTP) [18] to manage IRC network connections.

The web framework for this implementation is Phoenix web framework [19] in Elixir [20], and OTP [18] is exposed mainly as the generic server module [21] in the Elixir programming language [20]. In this specific implementation, with various methods of inter-process communication [22], OTP [18] keeps spawned connections up and running, and restarts stale connections.

1.4.3 WebSocket-Native User Interface

As a recent web framework, Phoenix [19] has adopted WebSocket [23] based web UI. Expensive computation is done on the server side and the data exchange between the server and the user interface is kept to the minimum.

Chapter 2

Background

2.1 Research

Though nowadays the concept of IRC [1] is seen in many places on the Internet, IRC [1] communications are mostly bridged. For example, on Discord, many communities use bot accounts to bridge IRC [1] channels so that they can chat with their members on IRC [1] seamlessly. This, however, imposes concerns for privacy and usability. First of all, all the bridged messages will go through the owner of the bot, which might contain sensitive information such as SASL [24] passwords. Secondly, it is not straightforward or at least not deterministic to communicate with the bridged IRC networks via the bridge bots, as certain IRC [1] commands or messages might be dropped silently.

There are no proper graphical user interface (GUI) clients for IRC [1] similar to modern web browsers such as Google Chrome [25] for HTTP. The reason is that the IRC [1] protocol is fairly constrained and not as extensible as HTTP for such a global Internet scale as today's. But the fact that IRC [1], the protocol, is simple enough, combined with the certainty that all IRC [1] servers communicate with their clients in the same way, makes it very simple to create an IRC [1] client from scratch. There are many command-line and GUI clients for IRC [1] in the free and open-source software sphere, but they either are hard to use or have to be used with a bouncer software so the chat history can be preserved properly and so on.

This study is dedicated to creating a web-based implementation that does not trouble the users with installation procedures, chat history management and so on. In the next section, a few similar projects are examined.

2.2 Competition

The research done above shows that even though there are countless IRC [1] clients available now and throughout the years, all of them require the user to possess some debugging skills or some understanding of the IRC [1] protocol per se. However, there are some user-centered solutions from recent years, but none of them are feature-complete, or require paid subscriptions for most basic functionalities.

- Convos

  Convos [7] is libre software. However, virtual private servers (VPS's) [26] are not free, and it might require a Linux expert to set up this software.

  As for actual usage, there are also some disadvantages.

  - The web interface is responsive but not very intuitive, because some terminologies on the UI such as connection, connection profile and conversation, are created, which even an experienced IRC [1] user still needs some time to get familiar with.
  - The configuration is not complete. For example, for set-ups with reverse proxy [27], it is still not possible to keep some attack vectors [28] in check.
  - The back-end is programmed in Perl [29] and some of the libraries are not very actively maintained.

  Overall, Convos [7] is only suitable for people who are interested in participating in the actual development of Convos [7].

- KiwiIRC

  This is a pure browser side solution, acting as a translation layer between HTTP and IRC.

  KiwiIRC [30] is used by various IRC [1] networks such as Libera.Chat [14] as the front-end solution as it does not consume any more resource from the server side apart from static web page hosting.

  The downsides are very obvious with this approach. The capability of this client relies entirely on browser side JavaScript and a connected state cannot be preserved if the user closes the browser tab.

- The Lounge

  The back-end of The Lounge [6] runs on the Node.js [31] run-time and is being actively maintained, and the front-end is designed with mobile users and extensibility in mind. There are full gesture support for smart phones and tablets, and input shortcuts for nicknames [10] and emojis. The communication between the web client and the back-end server can go through regular XHR [32] polling or WebSocket [23].

  However, The Lounge [6] only supports SQLite [33] as its database, which makes it difficult to scale and the front-end is relatively heavy on resource usage.

- IRCCloud

  IRCCloud [8] is a proprietary service, but it does have a lot of functionalities and native clients on iOS and Android, which makes IRC [1] function just like a lightweight Discord. The unpopularity might be because of the relatively high monthly subscription fee [34].

The actual product of this study, however, will not rely on promotions or advertisements but try to optimize the server-side source code and resource usage, so with less computing power more users can be served.

2.3 Other Inspiration

- Gajim

  Gajim [35] is a GUI client for XMPP [36], and excels in extensibility. It has a powerful plugin system where unofficial functionalities can be easily implemented and installed. This has minimized Gajim [35]'s code-base and chance of bugs.

- Matrix

  Matrix [37] is a new instant-messaging protocol with open REST [38] API and comprehensive documentation, so alternate clients can be implemented by the community.

  With a standardized or just solid implementation, a platform is productive and fun to build on.

- Jane Street

  Jane Street [39] is a trading company, and it is famous for using OCaml [40] as the main

programming language, but they are also working on making OCaml [40] structured more like Common Lisp [41], as "OCaml's metaprogramming story is kind of messy." [42]. It is true that OCaml [40] is mathematically more concise, but it is also too complex for engineering purposes. Selecting technologies that are fit to the task is crucial.

Chapter 3

Methods & Libraries

3.1 Method

The method used in this project is agile development. The initial design of this project tries to become a full translation layer between HTTP and IRC [1], which has imposed technical problems on user privacy and IRC [1] networks' moderation. Since then, the project started to focus on instant-messaging specific functionalities and modularity.

3.2 Libraries

The main library used for the prototype is Django web framework [43] and Phoenix web framework [19] is used for refactoring. This study tries to find out the simplest solution for IRC [1] to have a friendly user interface and to be integrated into the modern web. The simplest way is almost always difficult to find, so it is important to have a bare bone but less performant prototype to experiment with. As the server-client interactions become clearer, the back-end server codebase will be refactored with the Elixir [20] programming language using Phoenix web framework [19].

The initial prototype, however, will also be preserved and optimized using Python's foreign function interface (FFI) [44] with libraries such as PyO3 [45] and CFFI [46].

3.3 Aims & Objectives

IRC [1] networks are not as accessible as websites, because the evolution of the Internet has been concentrated on popularity. IRC [1], on the other hand, has stayed simple [47]. The idea of this project is to make IRC [1] networks more accessible while keeping it minimal and decentralized [48]. The aim is to create a web IRC [1] platform mainly for beginners and also catering to the needs of advanced users as the functionalities and code base become stable.

This project establishes, or tries to establish a way for certain software, such as IRC [1] servers, to communicate with the world using modern web technologies.

The main objective is to create a web server as a mediator between IRC networks and a modern responsive web front-end.
Additional objectives include evaluation and unit tests.

3.4 Requirements

User Requirements

Anyone who is interested in IRC [1] networks is a target user.

3.4.1 Requirement Specification

- Accessibility

  There are more than 400 [49] active IRC [1] networks on the Internet, and identity management could quickly become a pain if a user wants to access more than a few IRC [1] networks. For this scenario, it is better to have a unified and verified identity for multiple IRC [1] networks. This also boosts the user's reputation.

  On the other hand, IRC commands [50] are archaic and difficult to learn without related knowledge, so it's better to have a graphical approach.

- Discoverability

  IRC [1] networks are self-documented. Similar to OpenBSD [51], the documentation for IRC [49] is built-in on most of IRC [1] servers, but is usually ignored because of the lack

of intimacy. It is certainly better to make some IRC [1] channel recommendations to the users based on their interests and curiosity.

- Privacy

  Privacy has always been a concern for web applications. This project will use end-to-end encryption (E2EE) [52] as much as it is allowed to. For standard IRC [1] communications, advanced users will be able to use the application as an identity [24] provider for multiple IRC [1] networks.

  More privacy-related features will be added as the user base grows.

- Efficiency

  As modern computers are becoming a lot faster every year according to Moore's Law [53], many software developers have stopped caring about efficiency [54] of their source code, to the point where rendering some text might use more resource than playing a video file.

  But in reality, software efficiency is important. Efficient source code ensures the best user experience (UX) over average Internet connections on average devices, and reduces resource usage on the server-side, which in turn reduces server costs, and the price for end users will also be lower.

3.4.2 Functional Specification

- Identity Management

  The application needs to have a user identity system for itself and help existing users manage their IRC [1] network identities.

  Features:

  - Social Authentication

    The users should be able to directly log in to the application using OAuth2 [11] via various social sites such as Google [12], GitHub [13] and the official instance of GitLab [55].

  - User Session Management

The server should be able to store real-time user sessions for long-running connections and lower the frequency of re-authentication.

  - IRC Nick Registration

    The application should be able to automatically register nicknames [10] for every user at the first connection attempt to any consented IRC [1] network. A simple email inbox could be bundled to receive verification code or instructions.

  - User Credential Management

    The users' SASL [24] credentials could be stored on the server-side with end-to-end encryption (E2EE) [52]. The credentials' passphrases should be controlled by the users and decryption of stored credentials should happen on the client side.

- Discoverability Service

  This is an opt-in service. The application will recommend its users IRC [1] channels and friends. Registered users can consent to being advertized across the platform.

  Features

  - Group Chat Recommendation

    This is an opt-in feature. The application will be able to give each new user some recommended channels based on the user's interests.

  - Friend Recommendation

    This is an opt-in feature. The application will give users friend recommendations based on users' interests.

  - Friend Discovery

    This is an opt-in feature. After registration, a list of recommended friends will be generated for every user based on their interests.

- IRC Proxy

  Both the prototype and the production implementation are compatible with the IRC [1] protocol and will act as proxy between users and IRC [1] networks.

  Features

  - Basic IRC [1] Communication

These are basic IRC commands [50], such as querying users' information, listing existing group chats, and sending and receiving chat messages. The application will be fully compatible with these messages and commands, and offer more readable formatting for text content.

- XDCC [56]

  XDCC [56] is a file sharing method for IRC [1]. Although streaming XDCC [56] data with HTTP or WebSocket [23] is technically viable, it raises concerns for user privacy and copyright [57] infringement. Implementation of this feature has been stopped both on the prototype and the production implementation due to ethical considerations.

- Minimal Enhancements

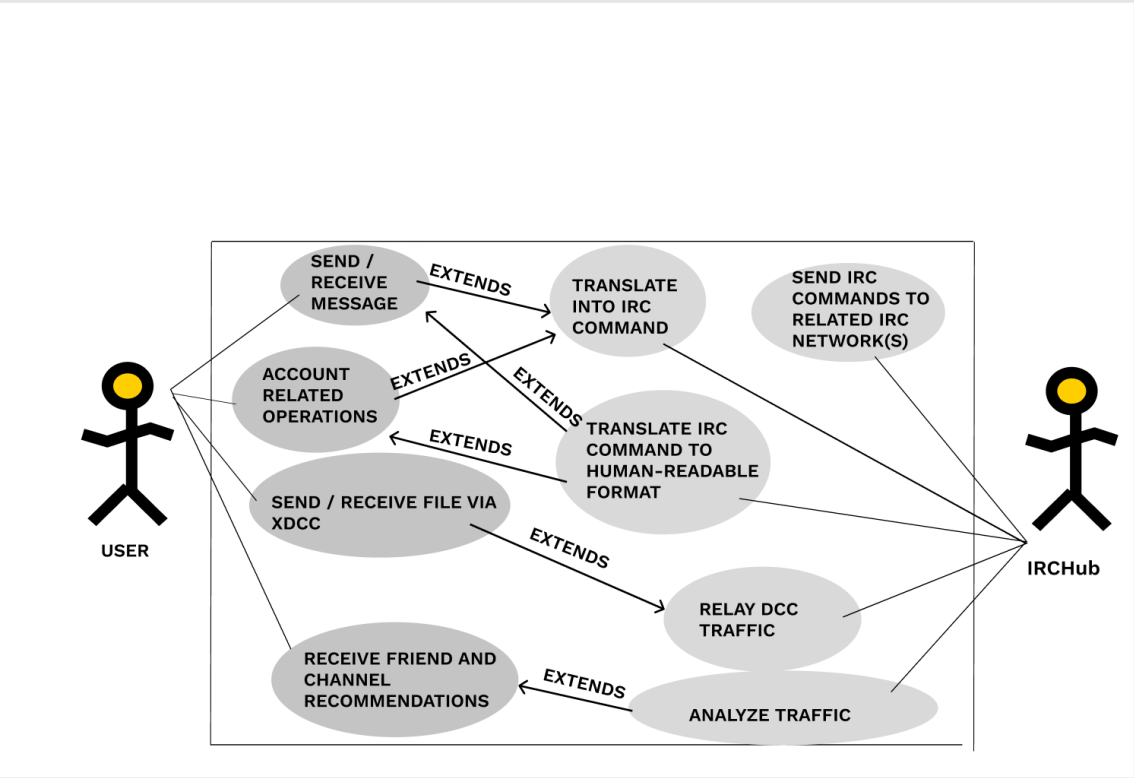
  The application will enhance on the IRC [1] protocol, for example, by providing additional pragmatical interfaces for chat history and global full-text search. However, these enhancements need to be simple and extensible, i.e., having the ability to interface with other software and protocols.

- REST [38] API

  The application's emphasis is at HTTP and the modern web. Both the prototype and the production implementation will provide REST [38] API endpoints for developers, so they do not have to start from scratch on developing IRC [1] applications.

3.5 System Design

3.5.1 Use Cases



- Chatting and Commands

  Users are able to use plain text, certain markup formats such as Markdown [58], or multimedia files to chat with other users just like on any other chat application. The difference is that this application utilizes an existing, proven-to-work and decentralized protocol that has existed since 1988 [59], 3 years before the introduction of HTTP [60].

- XDCC

  The application tries to support file sharing over XDCC [56] in the future. Even though it has significant flaws, XDCC [56] provides backward compatibility to archaic IRC [1] clients and utilities.

3.5.2 Pseudocode

- Initialize Nick

  This firstly authenticates the user from the HTTP request object, then loads and connects all linked IRC Nicks [10] to their corresponding networks. If some or all of the connections fail, an HTTP code 400 (Bad Request) will be returned, so the client will know there is some error and could attempt a retry.

```
user = IRCHubUser.from_request(request)
if user is None: # authentication failed
    return HttpCode(401)

nicks: Dict[network: nick] = IRCNick.multiple_from_request(user, request)
user.set_current_nicks(**nicks)

if None in nicks.values(): # Partially OK or all failed
    return HttpCode(400) # need to retry!
else
    return HttpCode(200) # OK
```

- Send Message

  The back-end server firstly retrieves the user and a network from the request object. Then it verifies that the user is indeed connected to the network. And then the server uses this user's identity to send a PRIVMSG command to the target. (PRIVMSG sends a message to a user or a group chat)

```
user = IRCHubUser.from_request(user)
network = request.network
if network not in user.networks.as_strings:
# Network not configured
    return HttpCode(400)

nick = user.get_nick(request.network)
if nick is None: # not currently logged in
```

```
        try:
            nick = IRCNick.login(user, network)
            except LoginFailed:
                return HttpCode(400) # still failing to login to network
    # format and escape message
    msg: str = corelibs.irc.quote_string(request.message)

    # send message via IRC
    error = nick.send_command("privmsg %s :%s", request.target, msg)
    if error is None: # OK
        return HttpCode(200)
    else # error occurred
        return HttpCode(400, error.to_string)
```

- Send File via XDCC (WebSocket)

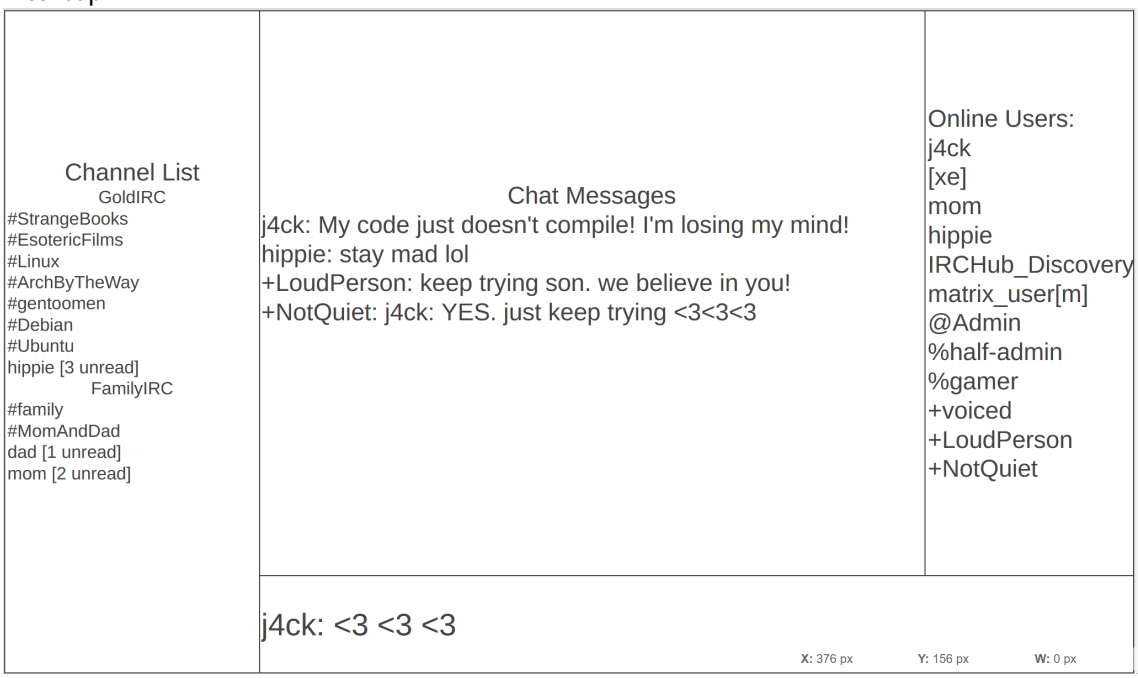
  This is a WebSocket [23] endpoint. After authentication, the server tries to acquire the file descriptor from the request. The file descriptor can be converted from a valid file URL or a filesystem path. Then the server runs a virus check and a copyright check on the file. If this file is good to be transferred, it calls xdcc_send_iterator to get the sender iterator(the "it" variable). The server sends a status update to the WebSocket [23] client for every successful iteration.

```
    user = IRCHubUser.from_request(request)
    fd = corelibs.file.from_request(request)
    try: # do some checks on the file
        extern.check_virus(fd)
        fd.seek(0)
        extern.check_copyright(fd)
        fd.seek(0)
    except HasVirus, CopyrightedFile:
        websocket.send({"error": "bad file", "file": fd.uuid})
        return
```

```
nick = user.get_nick(request.network)
it = nick.xdcc_send_iterator(request.target, fd)
for sent_bytes in it:
    percentage = sent_bytes / fd.size
    websocket.send(
    {
    "error": null,
    "progress": percentage,
    "file": fd.uuid
    }
    )
```

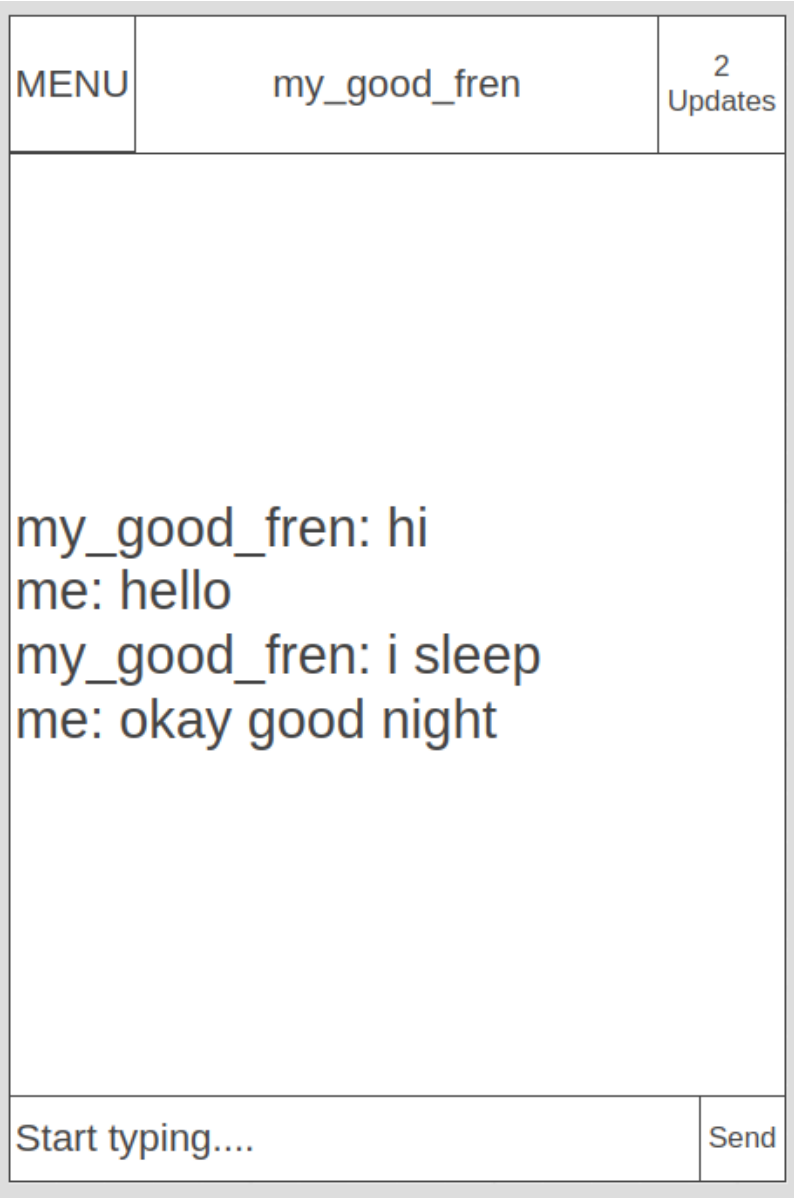
3.5.3 UI Wireframes

- Desktop



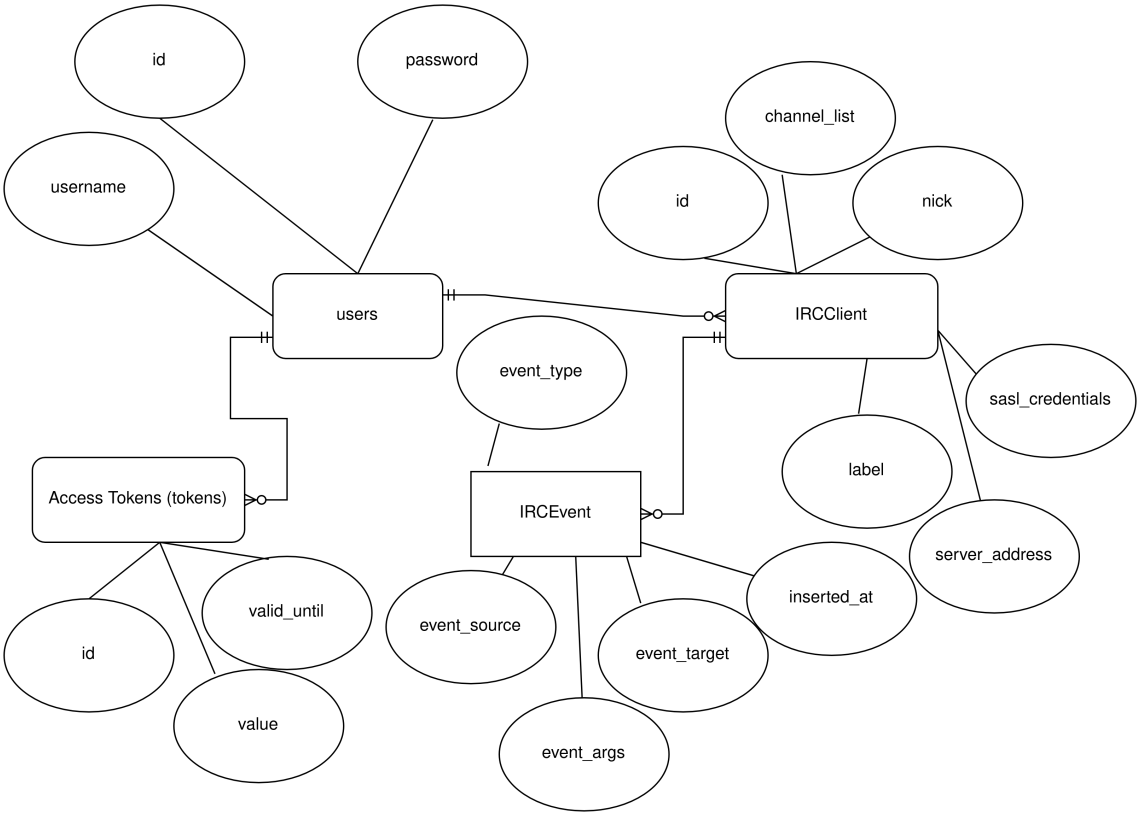
On the left is the IRC [1] channel list, and the member list is on the right with their current IRC [1] statuses. The main chat window is in the middle.

- Mobile



The menu on the top-left is a hamburger [61] menu, that is able to expand to a list of joined IRC [1] channels, user preference settings and general settings. On the top right is the notification icon, which is updated when the user is mentioned or there are other notifications. At the middle of the top bar is the chat target, which could be another user or an IRC [1] channel.

3.5.4 Database



Users' passwords are less important in the product model, since the focus is on social login options.

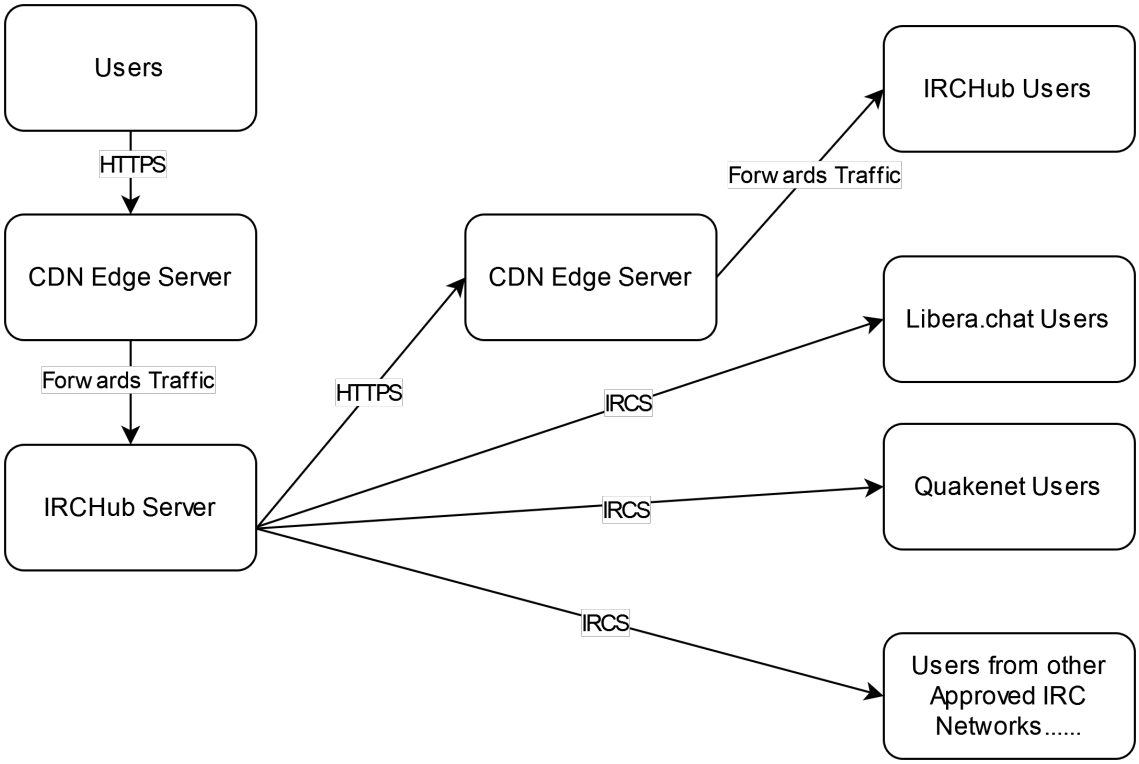
This makes the access token table essential for user authentication and authorization, and the `valid_until` field makes it easier to revoke a token to ensure maximal security.

The IRC [1] client table stores connection settings and each client's current identity, whose raw message exchange with target IRC [1] networks will be recorded in the IRC event [62]] table.

The IRC event [62] table is a direct mapping of the IRC event [62] class from jaraco's IRC library [63] for Python.

3.5.5 System

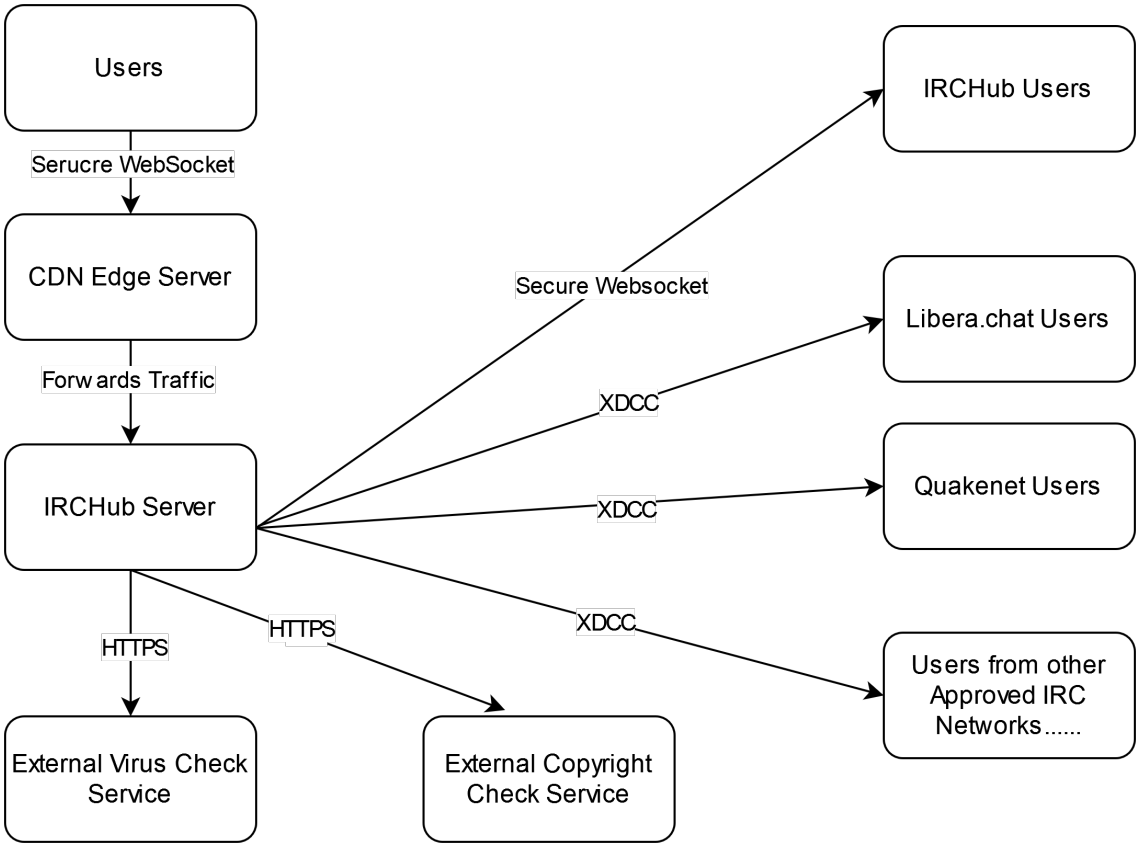
IRC



Content delivery network (CDN) edge servers will be deployed for better connection latencies.

The message routing mechanisms are explained in the System Design section.

XDCC Proxy



Content delivery network (CDN) edge servers with WebSocket [23] support could be deployed for faster XDCC [56] file sharing. The shared file will be routed in the manner described in the System Design section.

3.6 Development Process

3.6.1 Core Technologies

Django Web Framework

Django [15] has been a web framework known for rapid development. For the prototype, the development efficiency is further improved via a few hacks [64] here and there.

Notable hacks in the prototype include serving the front-end files directly from a Django [15] view, and using gevent [65] to achieve real multithreading [66], as is shown below.

```python
import datetime
import os
from dataclasses import asdict

from django.conf import settings
from django.http import FileResponse, Http404
from django.utils import timezone
from django.views import View
from rest_framework.response import Response
from rest_framework.views import APIView

from core.models import ChannelInfo, IRCClient, IRCEvent


class FrontendView(View):
    frontend_dir = settings.BASE_DIR / "frontend"

    def get(self, request, path="./", *args, **kwargs):
        path = path.lstrip("/")
        path = os.path.join(self.frontend_dir, path)
        if os.path.isdir(path):
            path = os.path.join(path, "index.html")
        if os.path.exists(path) is False:
            raise Http404("404 Not Found")
        file = open(path, "rb")
        return FileResponse(file)
```

Computer Networks

Some libraries in this project have rough edges, because there are not enough users and developers for these libraries. Lots of prior knowledge is required to use those libraries correctly.

Web UI Development

Vanilla JavaScript for browser-side development is simple but not a good practice in this day and age.

3.6.2 Development

Minimal Prototype

A minimal prototype has been developed to better demonstrate the complexity of the system. This was a very rapid process to get the application up-and-running.

Robust and Suitable Architecture

The application will eventually maintain multiple IRC [1] connections for many users, where every connection is a thread or a process. Python [16]'s GIL [67] makes multithreading [66] pointless, where Erlang [68] or Elixir [20] is good at multiprocessing [66] and utilization of

hardware resources. Phoenix web framework [19] made with Elixir [20] will be used to refactor the prototype implementation.

3.6.3 Testing

Manual Tests

Before systematic tests, regular users will be able to test the application's stability.

Unit Tests

This will be unit tests for source code and REST [38] API endpoints.

3.6.4 Evaluation

Feature Evaluation

Implemented features will be tested for their stability and completeness.

Real World Evaluation

Development deployments will be available publicly for evaluation.

3.7 Timeline

- 2023.10 - 2023.11
  1. Search for IRC [1] related Python libraries.
  2. Select the most suitable IRC [1] library.
  3. Start to work on REST [38] API using Django REST framework (DRF) [17].
- 2023.11 - 2023.12
  1. Make a simple but working web UI.
  2. Integrate the web UI with the REST API [38] and DRF [17] token authentication.
  3. Make sure that the prototype works.
- 2024.1 - 2024.2
  1. Introduce Phoenix web framework [19].
  2. Tweak bundled HEEx [69] template and add authentication functionalities to page

routing and so on.

3. Research on IRCv3 [4] capabilities.
4. Design and program the Open Telecom Platform (OTP) [18] application.

- 2024.2 - 2024.3
  1. Integrate the OTP [18] with the browser-side code.
  2. Integrate the browser-side user interface with Phoenix [19]'s PubSub [70] module and implement partial refresh.

- 2024.3 - 2024.4
  1. Integrate social login.
  2. Evaluation.
  3. Modify bundled unit test templates for the present architecture.
  4. Necessary documentation.

- 2024.4 - 2024.5
  1. Poster design
  2. Finalize the report and the codebase.

3.8 Risks and Risk Management

3.8.1 Unfamiliar Technology

Description

Presently it is rare to use vanilla JavaScript to make browser-side UI. Later this will be replaced by HEEx [69] templates generated by Phoenix web framework [19], which uses Tailwind CSS [71] for styling.

Management

New utilities and frameworks are introduced one at a time in order to make the cognitive load manageable.

3.8.2 Project Complexity

Description

With the simplicity and straightforwardness of the IRC [1] protocol, there are too many approaches to develop with it. Over time, the project could become too complex.

Management

Functionalities are introduced individually, and the project will try not to optimize the source code at an early stage.

3.9 Contingency Plan: Reuse the Prototype

In case the project becomes too complex or too hard to continue, the final implementation will reuse the prototype and implement an easier version that might require some workarounds.

Chapter 4

Results: Prototype with Django Web Framework

A prototype has been made with the Django web framework [15]. This prototype focuses on core functionalities, such as parsing IRC messages [62], converting plain text messages into JSON [72] responses and so on. And the web UI is made of HTML and vanilla JavaScript.

4.1 Code Repository

The source code repository is hosted publicly on GitHub under MIT License:

https://github.com/iatyl/final-project-prototype

The documentation is also on GitHub:

https://github.com/iatyl/final-project-prototype/wiki

4.2 Core Aspect

The core aspect of the prototype is to make a basic webchat platform that focuses on intimacy and quality.

4.3 Core Technology

4.3.1 Web Development

Web development is required for the web UI and the REST [38] API endpoints.

Here Django web framework [15] is used for the prototype, because Django [15] already has built-in user authentication, a dashboard for database visualization and so on. The focus is on the implementation of core functionalities.

4.3.2 Parallel Programming

In order to maintain multiple long-running IRC [1] connections, parallelism [66] is essential. And for the prototype, Python's parallelism [66] model is used despite the presence of GIL [73].

4.3.3 Network Programming

For the prototype, the `irc` [63] library is used as a wrapper for network programming interfaces. However, it is still important to be good at network programming for this task, because:

Firstly, the IRC [1] protocol, unlike HTTP, has not been fully adapted as native Python objects, so there will be inconsistencies that the programmer needs to be aware of.

On the other hand, the contributors of the `irc` [63] library assume that the users of this library are capable of basic network programming and the library itself has some rough edges. For example, SSL is broken [74] by default and a wrapper needed to be made for this prototype.

4.4 Features and Tests

4.4.1 Establishing IRC Connections

The back-end server should be able to store related user information and connect to an IRC [1] network on behalf of the user.

Explanation

This is the most basic functionality of the application. Once the connection is established, most other functionalities can be implemented by sending IRC [1] commands to the server, and parsing the server's responses.

Testing

Hands-on testing is required. Firstly the target server needs to recognize the prototype implementation as a connected client, then the application should be able to communicate with the server.

4.4.2 Basic Messaging Endpoints & UI

The backend server should be able to receive and send private chat messages to single users or group chats.

Explanation

This is the core feature.

Testing

This involves unit testing and manual testing. Manual tests ensure good user experience (UX) and unit tests are for the REST [38] endpoints.

4.4.3 Basic Client Status Endpoints & UI

The application should be able to fetch basic metadata, such as joined group chat list and member list, from the connected server.

Explanation

This is illustrated in the UI wireframe and gives the users more information.

Testing

This involves both manual testing and unit testing. The unit tests will be mainly for the REST [38] endpoints.

4.5 Additional Technology

4.5.1 Caching

The IRC [1] protocol focuses on real time communication between servers and clients, and can generate huge amounts of event [62] logs very quickly. Though many of these events [62] do not need to hit the persisted database, they are still very useful for a short while, so caching is a possible solution.

Risks

The risk here is the possibility that many of the events [62] are stored into the caching service and could never enter the database. This might cause data loss or data inconstancy. This approach requires more experiments.

4.5.2 Message Queues

There are many types of IRC events [50], and a message queue in front of the whole architecture is able to distribute these events by their types respectively. The application could be more modular and distributed this way.

Risks and Potential Difficulties

Even though message queues such as Apache Kafka [75] are used everywhere, using a message queue is not a normal practice and needs the developer to begin from scratch.

4.5.3 Phoenix LiveView

The prototype will be refactored using Phoenix web framework [19] when the proof-of-concept is clear. Phoenix web framework [19] is perfect at concurrency, and the Live View [76] components, in this scenario, can drastically reduce the resource usage on both the server side and the client side.

Risks and Potential Difficulties

Phoenix web framework [19] is very recent and making an IRC [1] application with it requires the developer to start from scratch.

4.5.4 IRCv3

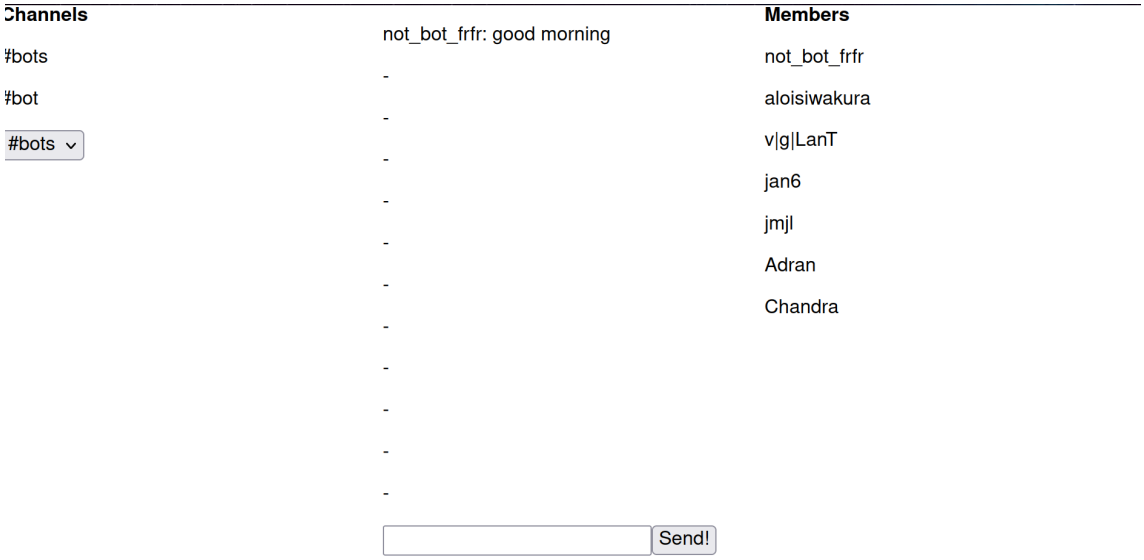
Implementing extra features as IRCv3 [4] extensions, such as `extended-join` [77], increases compatibility.

Risks and Potential Difficulties

Communicating ideas with other developers could be difficult, as the development of IRC [1] is rather scattered and focused on stability. Standardization is difficult to achieve.

4.6 Demonstration

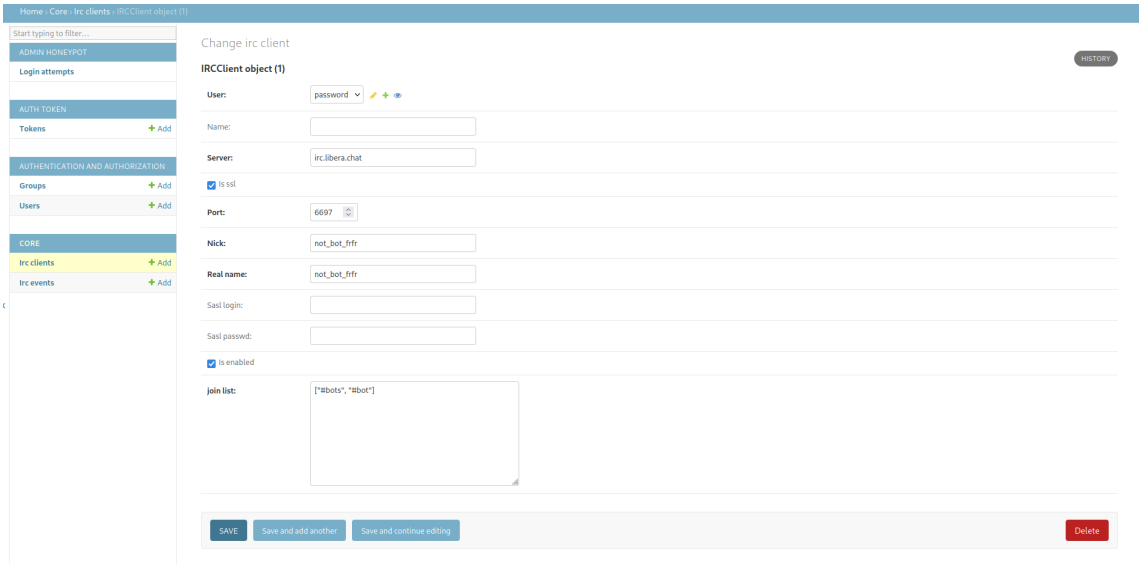
4.6.1 UI



The current front-end UI. This is planned to be improved later.

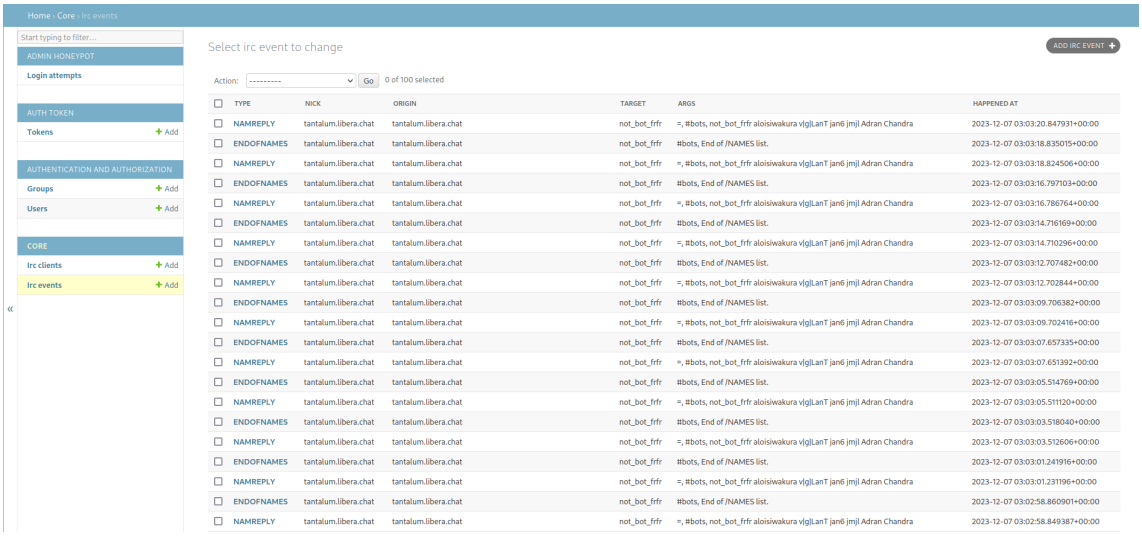
4.6.2 Database

Client



The fields for the database model of IRC [1] clients.

Events



TYPE	NICK	ORIGIN	TARGET	ARGS	HAPPENED AT
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:20.847931+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:18.835015+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:18.824506+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:16.797103+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:16.786764+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:14.716169+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:14.710296+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:12.707482+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:12.702844+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:09.706382+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:09.702416+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:07.657335+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:07.651392+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:05.514769+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:05.511120+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:03.518040+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:03.512606+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:03:01.241916+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:03:01.231196+00:00
ENDOFNAMES	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	#bots, End of /NAMES list.	2023-12-07 03:02:58.860901+00:00
NAMREPLY	tantalum.libera.chat	tantalum.libera.chat	not_bot_frfr	=, #bots, not_bot_frfr aloisiwakura v|g|LanT jan6 jmj| Adran Chandra	2023-12-07 03:02:58.849387+00:00

Sample real world IRC events [62].

4.6.3 Sample Code

Views

```
10          for client in enabled_clients:
 9              connection = client.get_connection()
 8              if connection is not None:
 7                  connected_clients.append(client.serialize())
 6          return Response({"clients": connected_clients})
 5
 4
 3 class ChannelMessageView(APIView):
 2     def post(self, request, *args, **kwargs):
 1         client_pk = request.data.get("client_pk")
96         channel = request.data.get("channel")
 1         offset_timestamp = float(request.data.get("offset_timestamp"))
 2         client = (
 3             IRCClient.objects.filter(is_enabled=True)
 4             .filter(user=request.user)
 5             .filter(pk=client_pk)
 6             .first()
 7         )
 8         if client is None:
 9             return Response({"messages": [], "error": "no such client"})
10         events = (
11             IRCEvent.objects.filter(client=client)
12             .filter(event_target=channel)
13             .filter(event_type="pubmsg")
14             .filter(
15                 inserted_at__gt=timezone.make_aware(
16                     datetime.datetime.fromtimestamp(offset_timestamp)
17                 )
18             )
19             .order_by("-inserted_at")
20             .all()
21         )
22         messages = [event.serialize() for event in events]
23         return Response({"messages": messages, "error": None})
INSERT  views.py
```

This is the view that is used by the browser to fetch chat messages. This view could be converted into a WebSocket [23] endpoint.

ObjectRelational Mapping (ORM) Operations

```
33
32      def channel_members(self, channel: str) -> t.List[str]:
31          cache_key = f"client-pk-{self.pk}-{channel}"
30          if channel not in self.join_list:
29              return []
28          cached_result = cache.get(cache_key)
27          if cached_result:
26              return cached_result
25          try:
24              self.command(f"NAMES {channel}")
23          except:
22              pass
21          time.sleep(0.5)
20          name_replies = (
19              IRCEvent.objects.filter(client=self)
18              .filter(event_type="namreply")
17              .order_by("-inserted_at")
16              .all()
15          )
14          for reply in name_replies:
13              if len(reply.event_arguments) != 3 or reply.event_arguments[1] != channel:
12                  continue
11              people = reply.event_arguments[2].split()
10              cache.set(cache_key, people, 100)
 9              return people
 8          return []
 7
 6      def command(self, raw_command: str):
 5          connection = self.get_connection()
 4          if connection is None:
 3              raise ValueError("could not get connection")
 2          connection.send_raw(raw_command)
 1
65      @property
NORMAL  models.py
```

The IRC [1] client model uses the `channel_members` method to get its current group chat's member list.

4.6.4 Tests

```
at 17:57:58 > pytest --verbose
Test session starts (platform: linux, Python 3.10.13, pytest 7.4.3, pytest-sugar 0.9.7)
cachedir: .pytest_cache
django: version: 4.2.6, settings: test.settings (from ini)
rootdir: /home/rei/Git/Goldsmiths/irchub
configfile: pytest.ini
plugins: Faker-21.0.0, sugar-0.9.7, django-4.7.0
collected 9 items

 core/tests/test_core.py::test_can_add_server ✓                                  11%
 core/tests/test_core.py::test_can_execute_irc_command ✓                         22%
 core/tests/test_core.py::test_can_fetch_member_list ✓                           33%
 core/tests/test_core.py::test_can_receive_chat_messages ✓                       44%
 core/tests/test_core.py::test_can_send_chat_messages ✓                          56%
 core/tests/test_core.py::test_can_save_chat_history ✓                           67%
 corelibs/tests/test_corelibs.py::test_async_all_of_can_fetch_all_objects ✓      78%
 corelibs/tests/test_corelibs.py::test_can_establish_irc_connection ✓            89%
 corelibs/tests/test_corelibs.py::test_generated_event_handler_can_handle_events ✓ 100%

Results (0.18s):
       9 passed
```

4.7 Additional Ethical Considerations

4.7.1 GDPR

The application stores users' chat history and other sensitive information such as users' interactions with various IRC [1] network servers.

4.7.2 The Path to GDPR

Besides general steps for web applications to achieve GDPR [78] compliance, the application will introduce additional end-to-end encryption (E2EE) [52] wherever possible for important user data.

Chapter 5

Results: Refactored Implementation

Due to Elixir [20] and Erlang [68]'s uncompromising approach to efficient parallelism, the project's architecture has become complicated as the amount of source code increase. Utilities such as the *Nix* package manager and the *Docker* containerization toolkit are used to manage the complexity. User privacy is a crucial aspect of this project. Some techniques for data protection and local regulation compliance are discussed in the Evaluation section, along with other aspects of the project.

During the development phase, some issues affecting user experience (UX) appeared after end-to-end testing. These technical issues and unit test templates provided by the Phoenix web framework [19] are explored in the End-to-End Testing and Unit Tests sections.

More technical details are discussed in the Technical Details & Technologies section.

5.1 Code Repository

The code repository is hosted publicly on GitHub under MIT License:

https://github.com/iatyl/goldsmiths-final-project

The documentation is also on GitHub:

https://github.com/iatyl/goldsmiths-final-project/wiki

5.2 Connection Map

The connection map is basically a hash-map of processes. Every process in these key-value pairs represents an active IRC [1] connection, and is able to communicate with other processes by means of inter-process communication(IPC) [22], using the functions provided by the generic server [21] module.

5.3 Utility Functions

Some utility functions are implemented for data structures such as property lists(plists) [79].

5.4 Configuration Dashboard

Similar to the Django Web Framework [43] which has a built-in admin dashboard [80], Phoenix web framework [19] provides customizable web views for managing user accounts [81] and other database models.

In addition to the bundled local IRC [1] server, users must add other servers on their own. Users will also be able to add revocable certificates for CertFP [82] in the future.

5.5 IRC Channel Hub

Users are able to chat in multiple IRC channels - chat group identifiers starting with "#" or "##".

5.6 Direct Messaging Page

Users are able to chat with each other and themselves, all users on the same IRC [1] server being identifiable by their IRC nicks [10].

5.7 Implementation Details

5.7.1 Connecting to the Database (PostgreSQL)

The database driver and library for this project is `ecto` [83] which is bundled with the Phoenix web framework [19].

Here is the user account schema generated using `ecto`:

(Full code is available at https://github.com/iatyl/goldsmiths-final-project/blob/main/lib/irchub/accounts/user.ex)

```
  schema "users" do
    field :email, :string
    field :password, :string, virtual: true, redact: true
    field :hashed_password, :string, redact: true
    field :confirmed_at, :naive_datetime

    timestamps(type: :utc_datetime)
  end
```

Here is the IRC client schema generated by `phoenix.gen.model`:

(Full code is available at https://github.com/iatyl/goldsmiths-final-project/blob/main/lib/irchub/chat/client.ex)

```
schema "clients" do
  field :channels, :string
  field :full_name, :string
  field :irchub_user_id, :integer
  field :is_enabled, :boolean, default: false
  field :nick, :string
  field :tag, :string
  field :url, :string

  timestamps(type: :utc_datetime)
end
```

The following UNIX shell command is used to migrate the database:

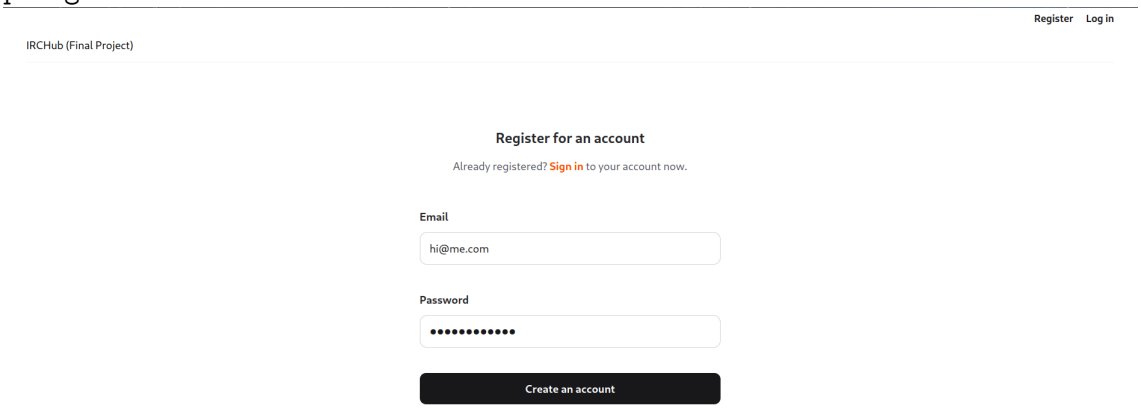
```
mix ecto.create && mix ecto.migrate
```

The output when migrating to a new PostgreSQL instance is available at Appendix C.

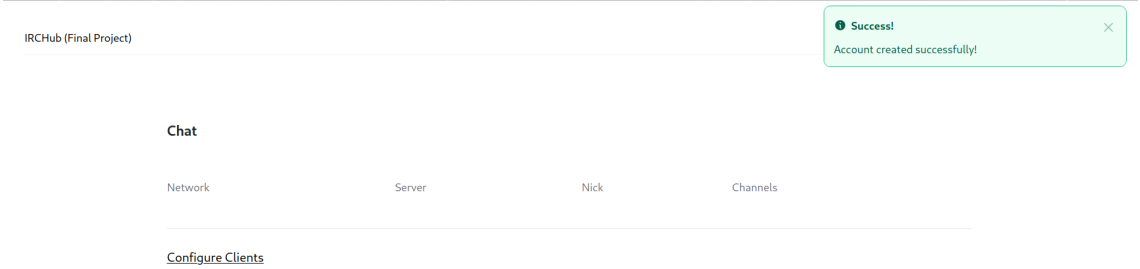
5.7.2 User Registration & Login

- Sign Up

  The easiest way to create a user account is by using the HTML views created with `phx.gen.auth`:



  Here a new user account is created, as is shown below:



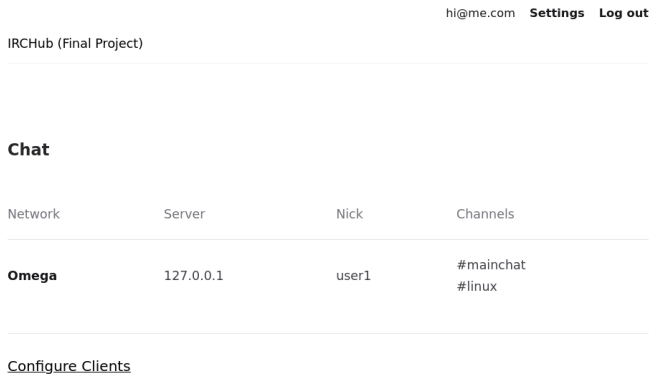
  User registration can also be done using the `iex` REPL [84], and is demonstrated in Appendix B.

- Login

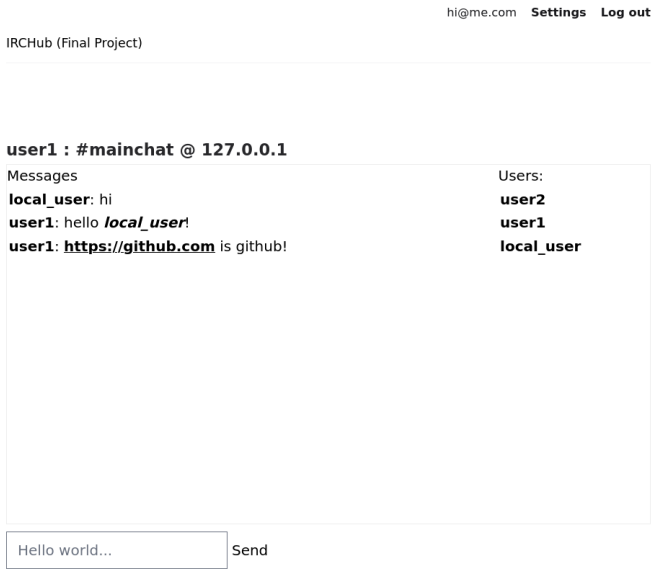
When a user has signed in using the web view, this user account's unique authorization token is shared among all WebSocket [23]-based Live Views [76], in addition to being saved in the web browser's cookie storage.

5.7.3 Group Chat

Users can select IRC [1] channels:

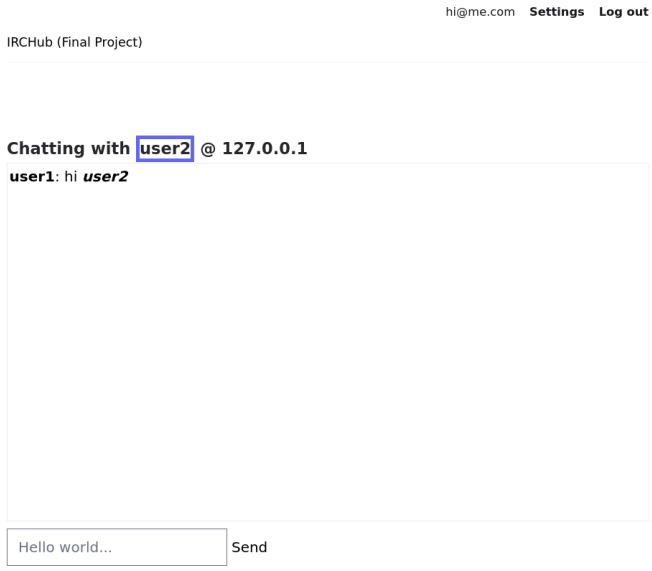


Here is the group chat page, where the IRC nicks [10] and URLs in the chat messages are highlighted:



5.7.4 Private Chat

Users are able to send direct messages to other users:



as well as themselves:

5.8 Chat Clients

The implementation of chat clients is an Open Telecom Platform(OTP) [18] application.

A hash-map of processes co-exists with the main supervision tree. This allows the internal applications to communicate with each other via the generic server (GenServer) [21] module.

5.9 Utilities

The Elixir programming language [20] and the Phoenix web framework [19] are both at an early stage. External utilities that are necessary for development setup and deployment are discussed in this section.

5.9.1 *Nix* Package Manager

Erlang OTP 25 or higher is needed for both development and deployment. *Nix* [85] can be used imperatively to set up the development environment (`-p` or `--packages` indicates what software *Nix* should provide for the current shell environment):

```
$ nix-shell -p erlang elixir stdenv inotify-tools direnv
```

In actual development, the following shell.nix file is used, for more detailed configuration.

```
...
pkgs.mkShell {
  buildInputs = [
  pkgs.erlang
  pkgs.cowsay
  pkgs.elixir
  pkgs.zsh
  pkgs.stdenv
  pkgs.inotify-tools
  pkgs.glibcLocales
  ];
    ...
```

5.9.2 *Docker* Containerization Toolkit

Docker [86] is able to spawn containers [87] from existing container images, which allows for quick deployment.

Docker Compose [88] is able to read container configuration from a Docker Compose [88] file and download needed container images. The following Docker Compose file is used to setup the PostgreSQL instance, ensuring it automatically starts along with the operating system and persisting its data in the `data` directory:

```yaml
services:
  db: # the name of the container
    image: postgres # use the postgres image for "db" container
    restart: unless-stopped # persist this container
    environment: # environment variables
      POSTGRES_PASSWORD: auf5eiliv7us2Fu3uQu4aimuy # randomized password
      PGDATA: /var/lib/postgresql/data/pgdata

    volumes: # directory list to be mounted into the container
      - './data/:/var/lib/postgresql/data/'
    ports:
      # only exposing the port locally
      - 127.0.0.1:5432:5432
```

Chapter 6

Discussion

Even though some HTTP API endpoints are implemented in the initial prototype, the Phoenix [19] implementation only supports browser-side usage. XDCC [56] is not implemented, either, due to technical complexity and ethical concerns.

This chapter is mainly about the production implementation in Phoenix [19] web framework. Testing & evaluation, technical details and challenges & difficulties are discussed.

6.1 Testing & Evaluation

6.1.1 End-to-End Testing

Currently the user interface is not good enough for daily use. Several issues were raised and are discussed in this section.

Issue: Mandatory E-Mail Address Verification

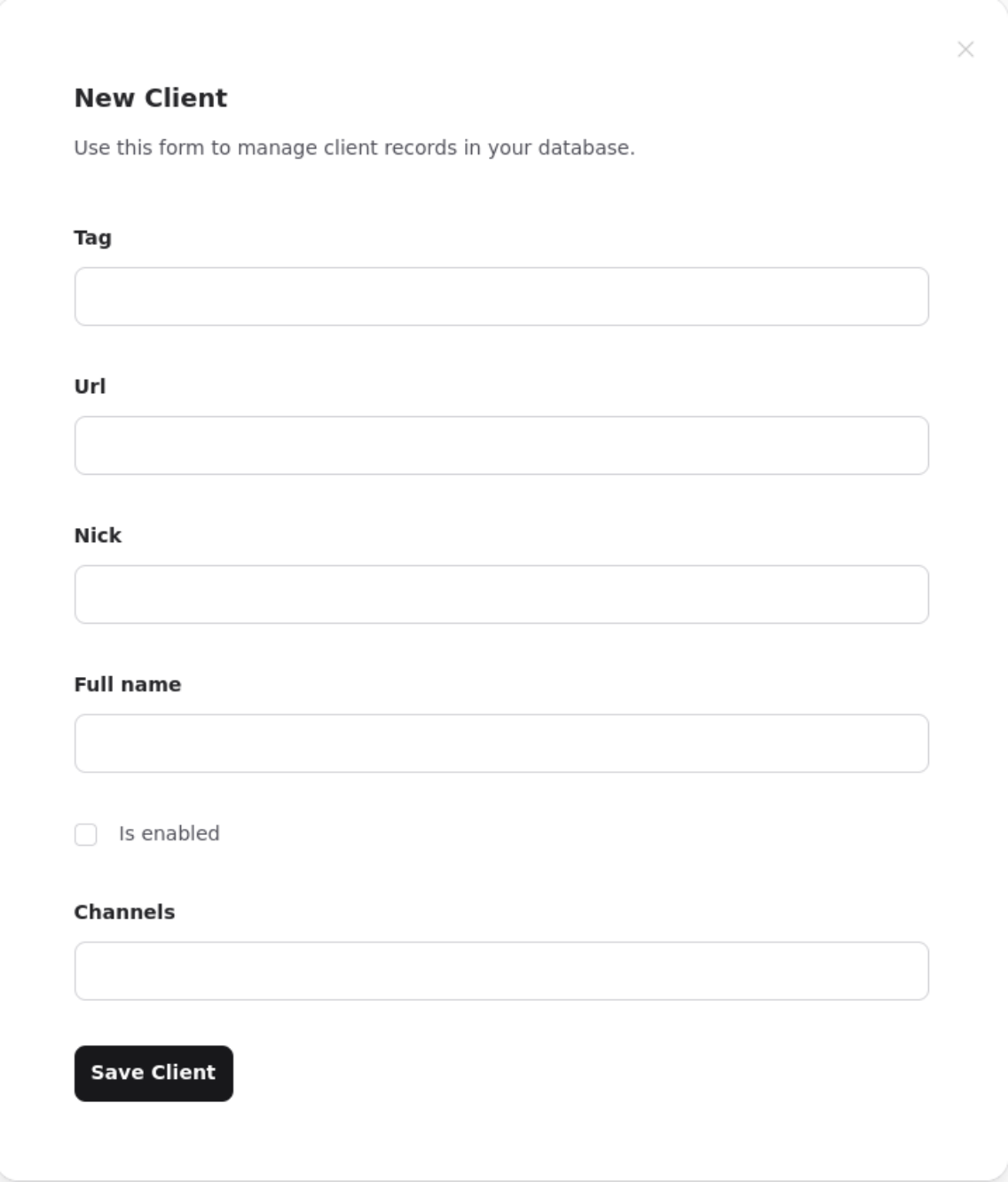
The Phoenix web framework [19] uses the *Swoosh* [89] library to deliver verification emails, but in such an early stage, email account verification should not be enforced. The following configuration in 'prod.exs' disables swoosh as well as mandatory E-Mail address verification during account registration:

```
# do not configure swoosh api client
config :swoosh, api_client, false
# use local storage
```

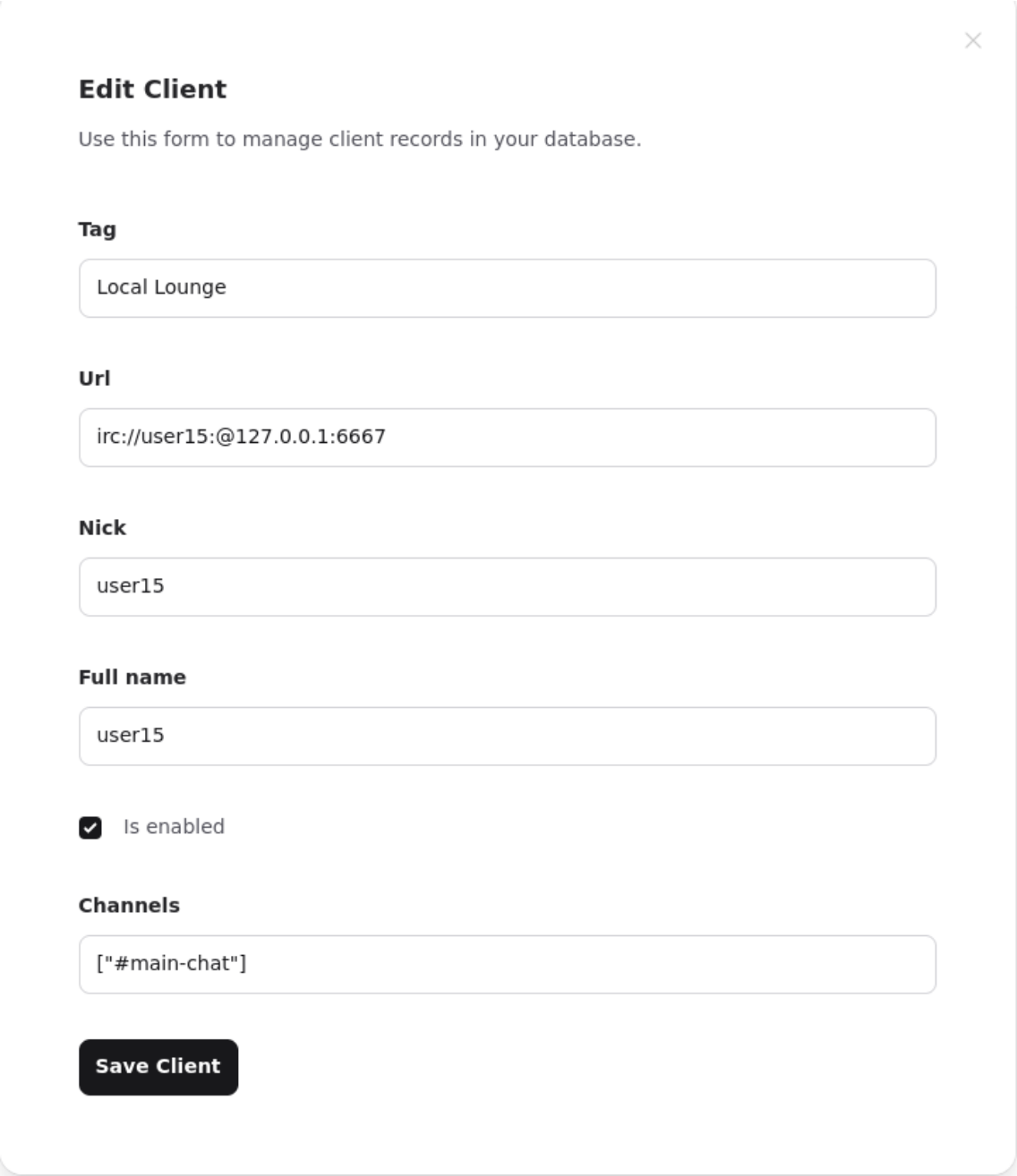
```
config :swoosh, local: true
```

Issue: Too Complicated to Configure

The client configuration view is not specific enough and overly technical as a graphical interface:



For now, this issue is addressed by automatically adding an example configuration item for every new user account, so the users are able to follow this example and add other IRC networks:



6.1.2 Unit Tests

Phx.gen.schema and *Phx.gen.live* also generate unit test templates in addition to database models and HTML templates. These unit tests include tests for Live Views [76] and for controllers [90].

However, many of the generated templates need to be modified as the main architecture is changed. For example, after user account registration, the web application will redirect the

web browser to the main page ("/hub/") instead of the index page ("/"). This requires the user account registration tests to either

- 1. match the HTTP redirection response:

```
# from `assert html =~ email`, to:
assert html =~ "/hub/"
```

- or 2. make the test follow HTTP redirects:

```
# from `{:ok, _lv, html} = live(conn, ~p"/users/log_in")`, to:
{:ok, _lv, html} = follow_redirect(conn, ~p"/users/log_in")
```

Presently, 21 out of 163 bundled unit tests are failing, as is shown in Appendix E.

6.1.3 Evaluation

Architecture

The Elixir programming language [20] and the nature of Open Telecom Platform (OTP) [18] require plenty of prior knowledge for open-source contributors.

Advantages

- Modular Code-Base

  The `mix phx.new` command generates very explicit template source code. Almost every part of the whole implementation can be modified without having multiple versions of dependencies.

- Simple Deployment

  The use of `nix` simplified deployments. With a properly configured reverse proxy, deploying the application is simply a one-liner shell command:

  `mix deps.get && mix setup && MIX_ENV=prod mix phx.server`.

- Interactive Development

  Because of the functional aspect of the Elixir programming language [20], the `iex` REPL [84] is able to be injected to the back-end server during initial setup, by using the `-S` option of the `iex` REPL [84]:

```
$ iex -S mix phx.server
[info] Running IrchubWeb.Endpoint with cowboy 2.10.0 at
↪  127.0.0.1:4000 (http)
[info] Access IrchubWeb.Endpoint at http://localhost:4000
Interactive Elixir (1.15.7) - press Ctrl+C to exit (type h() ENTER
↪  for help)
[watch] build finished, watching for changes...
iex(1)>
Rebuilding...
Done in 379ms.
nil
iex(2)>
```

  As is shown, the cowboy [91] HTTP server is started at port 4000 of `localhost` along with an interactive Elixir [20] shell. In this REPL [84], all Phoenix [19] structs and the database can be modified with Elixir [20] code.

Ways of Data Protection

Privacy is an important aspect of this implementation and this project in general. To achieve a usable status, the security of this implementation still has to be improved. Implementations and complying to local regulations are discussed in this section.

- Network Connections

  On a production deployment, all connections, including internal and external connections, should enforce Transport Layer Security(TLS) [92] encryption with trusted certificates and Perfect Forward Privacy [93].

- Password Expiration

  Every user with multi-factor authentication (MFA) [94] enabled should be able to opt in to the password expiration feature to invalidate the login password after long time of inactivity. Any multi-factor authentication (MFA) [94] method should be able to be used to reactivate the old password or set a new one.

- Other Techniques

  As new privacy standards are being published, more privacy features will be tested and included.

Implemented Data Protection Methods

At the moment, only a pop-up component for user consent of extra cookies is implemented.

- Pop-up for Unnecessary Cookies

  According to gdpr.eu, websites should "receive users' consent before you use any cookies except strictly necessary cookies." [95] A simple HEEx [69] component is placed at the top of the application to ask for users' consent for extra cookies, and then users' responses are stored locally at the browser-side and remotely at the server-side.

More on Data Protection

Regions such as the EU and the United Kingdom have strict privacy laws where adhering to regional regulations is a primary goal for improving user privacy. For regions where businesses' behaviors are less regulated, additional protection such as enforcing two-factor authentication should be added.

To conclude, however, not enough has been done regarding data protection and GDPR [78] compliance.

6.2 Technical Details & Technologies

6.2.1 Novel Approaches

In this section, two novel approaches to web development that are unique to the Phoenix web framework [19], dual-stack transport and HTML+EEx (HEEx) [69] template language, are discussed.

Dual-Stack Transport

The Phoenix web framework [19] uses both HTTP and WebSocket [23] as its native data transport methods, to ensure better user experience(UX) and maximal backward compatibility.

For example, if the WebSocket [23] handshake connection for Live Views [76] takes too long to finish, a browser will be considered not to support WebSocket [23], and data transport will be done with XMLHttpRequests (XHR) [32]

HTML+EEx (HEEx) Template Language

The HEEx [69] markup language is used to render the web pages for the Phoenix web framework [19]. As Live Views [76] are always synchronized with the server, techniques such as two-way binding [96] are achieved through WebSocket [23]. Most browser events, in addition to be exposed to the JavaScript interface, are sent to the corresponding Live View [76] controller [90], so these events will also be handled as server-side events.

In practice, the Phoenix web framework [19]'s approach is more robust and supports the observer pattern [97] but is less intuitive than strictly user-interface (UI) frameworks such as Angular [98].

6.2.2 ExIRC: An IRC Library for Elixir

ExIRC [99] is a flexible IRC [1] library to build with. Every IRC [1] connection is represented by a separate process on Bogdan's Erlang Abstract Machine (BEAM) [100] spawned by the generic server (GenServer) [21] module.
Sample code for connecting to an IRC server is available at Appendix D.

6.3 Challenges & Difficulties

Many of the IRC [1] building blocks, such as the `jaraco/irc` [63] library for Python [16] and ExIRC [99] for Elixir [20], are less well maintained and demanding developers to do their own research. The issue here is mainly about handling SSL or TLS [92], as the standard libraries of both Python [16] and Elixir [20] have gone through significant changes as TLS [92] standards progress.

In this section, challenges and difficulties of the implementations of both the initial prototype and the production implementation are discussed.

6.3.1 Different Programming Paradigms

Python and GIL

The `irc` [63] package from `pypi` [101] was used in the first prototype. Because of CPython [16]'s global interpreter lock (GIL) [67], this prototype could not achieve real parallelism with standard multi-threading libraries.

Even if better performance could be achieved with alternate ways of multiprocessing, C [102]-style syntax prevents Python [16] projects from efficient scaling.

Elixir and BEAM

Elixir [20]'s approach to multiprocessing leverages Bogdan's Erlang Abstract Machine (BEAM) [100]. On BEAM[100], the programming API for inter-process communication [22] is more robust and developer-friendly, and a BEAM [100] process is much more efficient in terms of hardware resource usage compared to a standard process on UNIX operating systems.

It is usual that a relatively small Elixir [20] application could spawn millions of BEAM [100] processes at runtime.

The Phoenix & Django Web Frameworks

Both Phoenix [19] and Django [43] are full-stack solutions. Phoenix web framework [19] was firstly released 10 years later than Django web framework and has been tightly integrated with WebSocket [23] related technologies.

For Django [43], the entry point is always `manage.py`, or the ASGI [103] or WSGI [104] application. It is easy to manage where and how the dependencies are imported. But Phoenix [19] is much more modularized and require some deeper understanding.

6.3.2 Incomplete Tool-Chain

It is difficult to install Erlang [68] OTP 25 on rolling-released [105] Linux platforms, some of which are transitioning to OTP 26. For example, the Erlang [68] version manager `kerl` often yielded errors on Arch Linux platforms, as demonstrated in Appendix A.

Nonetheless, Phoenix web framework [19] does not contain and cannot generate a sample environment variable configuration for production deployment. And the required environment variables are provided in the generated template in the `runtime.exs` file(GitHub link) file. For example, the environment variables named `SECRET_KEY_BASE`, `PHX_HOST` and `PORT` are needed for production setup:

```
...
  secret_key_base =
  System.get_env("SECRET_KEY_BASE") ||
      raise """
      environment variable SECRET_KEY_BASE is missing.
      You can generate one by calling: mix phx.gen.secret
      """

host = System.get_env("PHX_HOST") || "example.com"
port = String.to_integer(System.get_env("PORT") || "4000")
...
```

Chapter 7

Limitations and Future Works

7.1 Next Steps

7.1.1 Better Documentation & Tutorials

Both the Phoenix web framework [19] and IRC (defined by RFC1459 [1]) are not popular topics today. The web application implementation and the REST API would be much more accessible if good documentation and tutorials are in place.

7.1.2 Native Mobile Apps

Web applications, Flutter [106] applications and React Native [107] applications add plenty of extra hardware resource usage to the actual implementation. Native mobile apps built with Kotlin [108] or Swift [109] are able to drastically improve user experience(UX) on smartphones and other mobile devices.

7.2 Further Development

7.2.1 Better User Experience (UX)

The web user interface (UI) is now very technical even for experienced users. Further development of the Live Views [76] will be more focused on better user interactions and simplifying the configuration process.

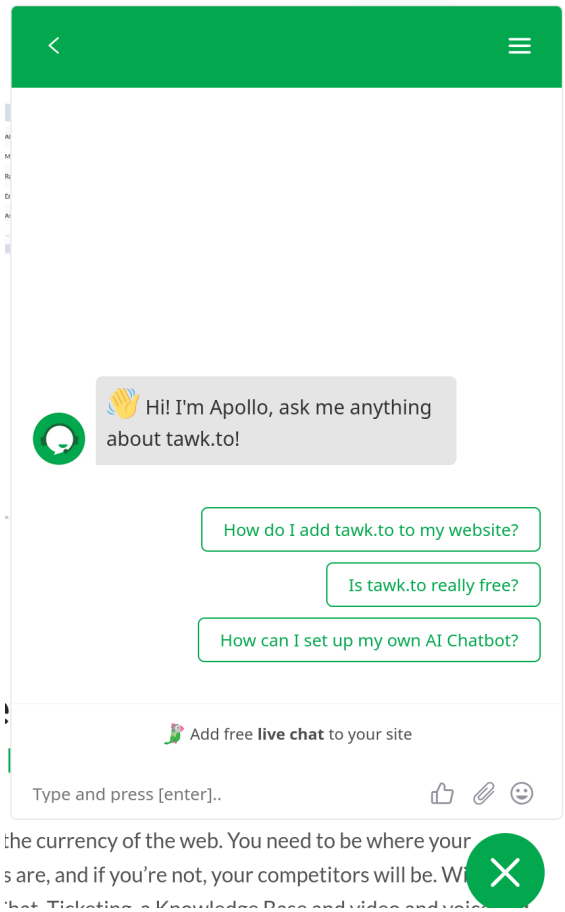
7.2.2 HTTP API

REST [38] based HTTP API was planned for this project and has only been partially implemented in the prototyping phase with DRF [17].

Further development of REST [38] API will firstly happen on the production implementation with Phoenix web framework [19] routers, along with a purely client-side web user interface (UI) to verify that the REST [38] API can be used for developing third-party clients.

7.2.3 Embeddable Chatbox Widget and Other Integratable Components

To popularize the application, it is important that developers are able to easily integrate the application to their websites or web apps. A possible component could be an integratable chat box, whose user interface (UI) is similar to `tawk.to` [110]:



However, in this project, users are able to chat with each other through an IRC [1] server configured by the website's owner.

As the application is open-source and will be better documented as time goes by, there will be

inspiration for more embeddable components, and the goal here is to be as flexible as possible.

7.2.4 Native Support for XMPP & Other Open Messsaing Standards

IRC [1] is a simple protocol as it uses an easy and human-readable format. For example, the raw IRC [1] command for sending the message "Hello, world!" to user "admin" is `PRIVMSG admin Hello, world!`, which is obviously not easily extendable. There are other open standards, such as Extensible Messaging & Presence Protocol (XMPP) [36] and Matrix [37].

XMPP [36] servers and clients use Extensible Markup Language (XML) [111] to exchange messages, and has standardized support for voice or video calling. However, it will not be an easy task to implement XMPP [36] features in Phoenix [19], even though production-ready libraries are available, such as the building blocks of `ejabberd` [112], an open-source XMPP [36] server in Elixir [20] and Erlang [68].

7.2.5 High Availability

High-availability does not come natural to IRC [1] networks. Modern solutions such as federating with ActivityPub [113] are powerful and fun to explore.

Chapter 8

Conclusion

IRC [1] has been backed by very competent free and open-source software communities for decades, and this project started from building an instant messaging platform based on the IRC [1] protocol, which has avoided a lot of "wheel reinventing" [114]. On the other hand, other interesting messaging protocols such as XMPP [36] also exist, and this project could implement those protocols as well.

The production implementation with Phoenix [19] was difficult. The Elixir/OTP [18] developer environment is highly modular and explicit, and the size of the codebase has been increasing rapidly. A possible solution is to define some macros [115] or a domain specific language (DSL) [116], as Elixir [20] is a functional programming language.

To conclude, scalable high parallelism [66] is not achieved without cost.

References

[1] J. Oikarinen et al. Internet Relay Chat Protocol, 1993. URL https://datatracker.ietf.org/doc/html/rfc1459. Accessed on 2023 Dec 12.

[2] freedesktop.org et al. Wayland: Wayland is a replacement for the X11 window system protocol and architecture with the aim to be easier to develop, extend, and maintain., 2024. URL https://wayland.freedesktop.org/. Accessed on 2024 Mar 24.

[3] ArchWiki Authors et al. Weston, 2024. URL https://wiki.archlinux.org/title/Weston. Accessed on 2024 Mar 24.

[4] IRCv3 Working Group. IRCv3: Open, extensible, feature-rich chat, proven through years of use., 2023. URL https://ircv3.net/. Accessed on 2023 Dec 13.

[5] Sadie Powell. chathistory Batch Type , 2015. URL https://ircv3.net/specs/batches/chathistory. Accessed on 2024 Apr 22.

[6] astorije et al. The Lounge: The self-hosted web IRC client, 2024. URL https://thelounge.chat/. Accessed on 2024 Apr 20.

[7] Convos. convos - A better chat experience, 2024. URL https://convos.chat/. Accessed on 2024 May 5.

[8] IRCCloud Ltd. IRCCloud - Group chat for teams, friends, and communities., 2024. URL https://www.irccloud.com/. Accessed on 2024 May 5.

[9] Microsoft. What is SaaS?, 2024. URL https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-saas/. Accessed on 2024 May 5.

[10] Wikimedia. Register your nickname, identify, and enforce, 2024. URL https://meta.wikimedia.org/wiki/IRC/Instructions#Register_your_nickname,_identify,_and_enforce. Accessed on 2024 May 5.

[11] Internet Engineering Task Force (IETF). The OAuth 2.0 Authorization Framework, 2012. URL https://datatracker.ietf.org/doc/html/rfc6749. Accessed on 2023 Dec 12.

[12] Google LLC. OAuth 2.0 for Client-side Web Applications. URL https://developers.google.com/identity/protocols/oauth2/javascript-implicit-flow. Accessed on 2023 Dec 12.

[13] GitHub, Inc. Authorizing OAuth apps. URL https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps. Accessed on 2023 Dec 12.

[14] Libera Chat's contributors. Libera.Chat, 2021. URL https://libera.chat/. Accessed on 2024 May 8.

[15] Wikipedia. Django (web framework), 2023. URL https://en.wikipedia.org/wiki/Django_(web_framework). Accessed on 2023 Dec 12.

[16] Python Software Foundation. python/cpython, 2024. URL https://github.com/python/cpython. Accessed on 2024 Apr 15.

[17] Encode OSS el al. Django REST framework, 2024. URL https://www.django-rest-framework.org/. Accessed on 2024 May 11.

[18] Fred Hebert. What is OTP? It's The Open Telecom Platform!, 2024. URL https://learnyousomeerlang.com/what-is-otp. Accessed on 2024 Mar 24.

[19] Wikipedia. Phoenix (web framework), 2023. URL https://en.wikipedia.org/wiki/Phoenix_(web_framework). Accessed on 2023 Dec 12.

[20] Wikipedia. Elixir (programming language), 2023. URL https://en.wikipedia.org/wiki/Elixir_(programming_language). Accessed on 2023 Dec 12.

[21] Elixir Contributors et al. GenServer, 2024. URL https://hexdocs.pm/elixir/1.16/GenServer.html. Accessed on 2024 Mar 24.

[22] Wikipedia et al. Inter-process communication, 2024. URL https://en.wikipedia.org/wiki/Inter-process_communication. Accessed on 2024 Mar 24.

[23] WHATWG (Apple, Google, Mozilla, Microsoft). WebSockets: Living Standard, 2024. URL https://websockets.spec.whatwg.org/. Accessed on 2024 Mar 28.

[24] Attila Molnar, William Pitcock. IRCv3.2 SASL Authentication, 2014, 2015. URL https://ircv3.net/specs/extensions/sasl-3.2. Accessed on 2023 Dec 12.

[25] Google LLC. chrome: The browser built to be fast, 2024. URL https://www.google.com/chrome/. Accessed on 2024 May 8.

[26] Google LLC. What is a Virtual Private Server (VPS)?, 2024. URL https://cloud.google.com/learn/what-is-a-virtual-private-server. Accessed on 2024 May 8.

[27] Cloudflare, Inc. What is a reverse proxy? | Proxy servers explained, 2024. URL https://www.cloudflare.com/en-gb/learning/cdn/glossary/reverse-proxy/. Accessed on 2024 May 8.

[28] Cloudflare, Inc. What is an attack vector?, 2024. URL https://www.cloudflare.com/en-gb/learning/security/glossary/attack-vector/. Accessed on 2024 May 8.

[29] Perl.org. Perl is a highly capable, feature-rich programming language with over 36 years of development., 2024. URL https://www.perl.org/. Accessed on 2024 May 8.

[30] prawnsalad et al. kiwiirc: Next generation of the Kiwi IRC web client, 2024. URL https://github.com/kiwiirc/kiwiirc. Accessed on 2024 May 8.

[31] OpenJS Foundation. node.js, 2024. URL https://nodejs.org/. Accessed on 2024 May 8.

[32] Individual mozilla.org Contributors. XMLHttpRequest, 2024. URL https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest. Accessed on 2024 May 8.

[33] SQLite Contributors. SQLite, 2024. URL https://www.sqlite.org/index.html. Accessed on 2024 May 8.

[34] IRCCloud Ltd. IRCCloud: Pricing, 2024. URL https://www.irccloud.com/pricing. Accessed on 2024 May 8.

[35] Gajim Developers. Gajim, 2023. URL https://gajim.org/. Accessed on 2024 May 8.

[36] XMPP Standards Foundation (XSF). XMPP: The universal messaging standard, 2024. URL https://xmpp.org. Accessed on 2024 May 8.

[37] The Matrix.org Foundation. matrix: An open network for secure, decentralised communication, 2024. URL https://matrix.org. Accessed on 2024 May 8.

[38] Red Hat, Inc. What is a REST API?, 2020. URL https://www.redhat.com/en/topics/api/what-is-a-rest-api. Accessed on 2023 Dec 12.

[39] Jane Street Group, LLC. Jane Street, 2024. URL https://www.janestreet.com/. Accessed on 2024 May 8.

[40] OCaml Contributors. OCaml, 2024. URL https://ocaml.org. Accessed on 2024 May 8.

[41] Common-Lisp.Net maintainers. Common-Lisp.net, 2019. URL https://common-lisp.net/. Accessed on 2024 May 8.

[42] Yaron Minsky. Extension Points, or how OCaml is becoming more like Lisp, 2014. URL https://blog.janestreet.com/extension-points-or-how-ocaml-is-becoming-more-like-lisp/. Accessed on 2024 May 8.

[43] Django Software Foundation et al. django: The web framework for perfectionists with deadlines., 2024. URL https://docs.djangoproject.com/en/dev/ref/contrib/admin/. Accessed on 2024 Mar 24.

[44] Wikipedia. Foreign function interface, 2024. URL https://en.wikipedia.org/wiki/Foreign_function_interface. Accessed on 2024 May 8.

[45] davidhewitt et al. The PyO3 user guide, 2024. URL https://pyo3.rs/v0.21.2/. Accessed on 2024 May 8.

[46] Armin Rigo, Maciej Fijalkowski. CFFI documentation, 2024. URL https://cffi.readthedocs.io/en/latest/. Accessed on 2024 May 8.

[47] Susam Pal. Simplicity of IRC, 2022. URL https://susam.net/maze/simplicity-of-irc.html. Accessed on 2023 Dec 12.

[48] Drew DeVaul. Absence of certain features in IRC considered a feature, 2019. URL https://drewdevault.com/2019/07/01/Absence-of-features-in-IRC.html. Accessed on 2023 Dec 12.

[49] netsplit.de. IRC Networks - in alphabetical order, 1998 -. URL https://netsplit.de/networks/. Accessed on 2023 Dec 12.

[50] J. Oikarinen et al. Server queries and commands - Internet Relay Chat Protocol, 1993. URL https://datatracker.ietf.org/doc/html/rfc1459#section-4.3. Accessed on 2023 Dec 12.

[51] OpenBSD Project. man display manual pages, 2022. URL https://man.openbsd.org/man. Accessed on 2023 Dec 12.

[52] IBM. What is end-to-end encryption? URL https://www.ibm.com/topics/end-to-end-encryption. Accessed on 2023 Dec 13.

[53] Intel Corporation. Moores Law, 2023. URL https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html. Accessed on 2023 Dec 12.

[54] John Spacey. 6 Examples of Software Bloat, 2023. URL https://simplicable.com/design/software-bloat. Accessed on 2023 Dec 12.

[55] GitLab Inc. OAuth 2.0 identity provider API. URL https://docs.gitlab.com/ee/api/oauth2.html. Accessed on 2023 Dec 12.

[56] Wikipedia. XDCC, 2023. URL https://en.wikipedia.org/wiki/XDCC. Accessed on 2023 Dec 12.

[57] U.S. Copyright Office. THE DIGITAL MILLENNIUM COPYRIGHT ACT OF 1998, 1998. URL https://www.copyright.gov/legislation/dmca.pdf. Accessed on 2023 Dec 12.

[58] The Daring Fireball Company LLC. . Markdown, 2004. URL https://daringfireball.net/projects/markdown/. Accessed on 2023 Dec 12.

[59] Wikipedia. Internet Relay Chat, 2023. URL https://en.wikipedia.org/wiki/Internet_Relay_Chat. Accessed on 2023 Dec 12.

[60] Wikipedia. HTTP, 2023. URL https://en.wikipedia.org/wiki/HTTP. Accessed on 2023 Dec 12.

[61] Wikipedia. Hamburger button, 2023. URL https://en.wikipedia.org/wiki/Hamburger_button. Accessed on 2023 Dec 12.

[62] J. Oikarinen et al. RFC1459:message-details, 2024. URL https://www.rfc-editor.org/rfc/rfc1459#section-4. Accessed on 2024 May 11.

[63] Jason R. Coombs et al. irc: Full-featured Python IRC library for Python. , 2023. URL https://github.com/jaraco/irc. Accessed on 2023 Dec 12.

[64] Wikipedia. Hacker culture, 2023. URL https://en.wikipedia.org/wiki/Hacker_culture. Accessed on 2023 Dec 13.

[65] gevent contributors . What is gevent?, 2023. URL https://www.gevent.org/. Accessed on 2023 Dec 13.

[66] Wikipedia. Parallel computing, 2024. URL https://en.wikipedia.org/wiki/Parallel_computing. Accessed on 2024 May 11.

[67] eriky et al. GlobalInterpreterLock, 2020. URL https://wiki.python.org/moin/GlobalInterpreterLock. Accessed on 2024 Mar 28.

[68] Wikipedia. Erlang (programming language), 2023. URL https://en.wikipedia.org/wiki/Erlang_(programming_language). Accessed on 2023 Dec 12.

[69] chrismccord et al. Components and HEEx, 2024. URL https://hexdocs.pm/phoenix/components.html. Accessed on 2024 Apr 16.

[70] chrismccord et al. Phoenix.PubSub, 2024. URL https://hexdocs.pm/phoenix_pubsub/Phoenix.PubSub.html. Accessed on 2024 May 13.

[71] Tailwind Labs Inc. tailwindcss: Rapidly build modern websites without ever leaving your HTML., 2024. URL https://tailwindcss.com/. Accessed on 2024 May 13.

[72] Mozilla. Working with JSON, 2024. URL https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON. Accessed on 2024 May 13.

[73] Abhinav Ajitsaria. What Is the Python Global Interpreter Lock (GIL)? URL https://realpython.com/python-gil/. Accessed on 2023 Dec 12.

[74] Bryan Davis . irc.connection.Factory.connect should support ssl.SSLContext.wrap_socket with the server_hostname parameter, 2023. URL https://github.com/jaraco/irc/issues/216. Accessed on 2023 Dec 13.

[75] Apache Software Foundation. Apache Kafka: More than 80% of all Fortune 100 companies trust, and use Kafka. , 2023. URL https://kafka.apache.org/. Accessed on 2023 Dec 13.

[76] chrismccord et al. Phoenix.LiveView, 2024. URL https://hexdocs.pm/phoenix_live_view/Phoenix.LiveView.html. Accessed on 2024 Mar 24.

[77] Kiyoshi Aman. extended-join Extension, 2011. URL https://ircv3.net/specs/extensions/extended-join. Accessed on 2023 Dec 13.

[78] The European Parliament and the Council. General Data Protection Regulation, 2016. URL https://gdpr-info.eu/. Accessed on 2023 Dec 13.

[79] Guy L. Steele, Thinking Machines, Inc. 10.1. The Property List - Common Lisp the Language, 2nd Edition, 1990. URL https://faculty.cc.gatech.edu/~isbell/references/lisp/clm/node108.html. Accessed on 2024 Mar 24.

[80] Django Software Foundation et al. The Django admin site, 2024. URL https://docs.djangoproject.com/en/dev/ref/contrib/admin/. Accessed on 2024 Mar 24.

[81] Berenice Medel (@bemesa21). Bringing Phoenix Authentication to Life , 2022. URL https://fly.io/phoenix-files/phx-gen-auth/. Accessed on 2024 Mar 24.

[82] Libera Chat's contributors. Using CertFP, 2021. URL https://libera.chat/guides/certfp. Accessed on 2024 Mar 24.

[83] josevalim et al. Ecto Documentation, 2024. URL https://hexdocs.pm/ecto/Ecto.html. Accessed on 2024 Mar 24.

[84] Jeanelle Horcasitas. What Is REPL?, 2021. URL https://www.digitalocean.com/community/tutorials/what-is-repl. Accessed on 2024 Mar 31.

[85] NixOS contributors et al. Nix & NixOS, 2024. URL https://nixos.org. Accessed on 2024 Mar 24.

[86] Docker Inc. Get Docker: Docker is an open platform for developing, shipping, and running applications., 2024. URL https://docs.docker.com/get-docker/. Accessed on 2024 Apr 20.

[87] Google LLC. What are Containers?, 2024. URL https://cloud.google.com/learn/what-are-containers. Accessed on 2024 Apr 20.

[88] Docker Inc. Docker Compose overview, 2024. URL https://docs.docker.com/compose/. Accessed on 2024 Apr 15.

[89] stevedomin et al. swoosh: Compose, deliver and test your emails easily in Elixir. Supports SMTP, Sendgrid, Mandrill, Postmark, Mailgun and many more out of the box. Preview your emails in the browser. Test your email sending code., 2024. URL https://hex.pm/packages/swoosh/. Accessed on 2024 Mar 31.

[90] MDN contributors. MVC: The Controller, 2024. URL https://developer.mozilla.org/en-US/docs/Glossary/MVC#the_controller. Accessed on 2024 Apr 20.

[91] Loïc Hoguin. Cowboy: Small, fast, modern HTTP server, 2018. URL https://ninenines.eu/. Accessed on 2024 Apr 1.

[92] Cloudflare, Inc. What is TLS (Transport Layer Security)?, 2024. URL https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/. Accessed on 2024 Apr 20.

[93] Sectigo Limited. Perfect Forward Secrecy Explained, 2024. URL https://www.sectigo.com/resource-library/perfect-forward-secrecy. Accessed on 2024 Apr 20.

[94] Cloudflare, Inc. What is multi-factor authentication (MFA)?, 2024. URL https://www.cloudflare.com/learning/access-management/what-is-multi-factor-authentication/. Accessed on 2024 Apr 20.

[95] Proton AG. Cookies, the GDPR, and the ePrivacy Directive: Cookie compliance, 2024. URL https://gdpr.eu/cookies/. Accessed on 2024 May 11.

[96] Google LLC et al. Two-way binding, 2024. URL https://angular.io/guide/two-way-binding. Accessed on 2024 Apr 16.

[97] Wikipedia. Observer pattern, 2024. URL https://en.wikipedia.org/wiki/Observer_pattern. Accessed on 2024 May 15.

[98] Google LLC et al. Angular: Deliver web apps with confidence, 2024. URL https://angular.io/. Accessed on 2024 Apr 16.

[99] bitwalker et al. exirc: An IRC client library for Elixir., 2020. URL https://hex.pm/packages/exirc. Accessed on 2024 Mar 28.

[100] Björn Gustavsson. A Brief History of the BEAM Compiler: BEAM (Bogdans Erlang Abstract Machine), 2018. URL https://www.erlang.org/blog/beam-compiler-history/#beam-bogdans-erlang-abstract-machine. Accessed on 2024 Mar 28.

[101] Python Software Foundation. PyPI ů The Python Package Index, 2024. URL https://pypi.org/. Accessed on 2024 Apr 15.

[102] Wikipedia. C (programming language)), 2024. URL https://en.wikipedia.org/wiki/C_(programming_language). Accessed on 2024 Apr 22.

[103] Wikipedia. Asynchronous Server Gateway Interface, 2024. URL https://en.wikipedia.org/wiki/Asynchronous_Server_Gateway_Interface. Accessed on 2024 Apr 22.

[104] Wikipedia. Web Server Gateway Interface, 2024. URL https://en.wikipedia.org/wiki/Web_Server_Gateway_Interface. Accessed on 2024 Apr 22.

[105] Abhishek Prakash. Linux Jargon Buster: What is a Rolling Release Distribution?, 2023. URL https://itsfoss.com/rolling-release/. Accessed on 2024 Apr 20.

[106] Google LLC et al. Flutter: Build for any screen, 2024. URL https://flutter.dev/. Accessed on 2024 Mar 31.

[107] Meta Platforms, Inc. et al. React Native: Learn once, write anywhere., 2024. URL https://reactnative.dev/. Accessed on 2024 Mar 31.

[108] Google LLC. Kotlin: Develop Android apps with Kotlin, 2024. URL https://developer.android.com/kotlin/. Accessed on 2024 Mar 31.

[109] Apple Inc. Swift: The powerful programming language thats also easy to learn., 2024. URL https://developer.apple.com/swift/. Accessed on 2024 Mar 31.

[110] tawk.to, inc. tawk.to, 2024. URL https://www.tawk.to/. Accessed on 2024 May 11.

[111] W3C: Information and Knowledge Domain. Extensible Markup Language (XML), 2024. URL https://www.w3.org/XML/. Accessed on 2024 May 14.

[112] ProcessOne. ejabberd: Robust, Scalable and Extensible Realtime Platform XMPP Server + MQTT Broker + SIP Service, 2024. URL https://www.ejabberd.im/. Accessed on 2024 May 15.

[113] W3C. ActivityPub, 2024. URL https://www.w3.org/TR/activitypub/. Accessed on 2024 May 15.

[114] Elliot Williams. Reinventing The Wheel, 2021. URL https://hackaday.com/2021/05/08/reinventing-the-wheel/. Accessed on 2024 May 11.

[115] Elixir Contributors. Elixir: Macros, 2024. URL https://hexdocs.pm/elixir/main/macros.html. Accessed on 2024 May 11.

[116] Wikipedia. Domain-specific language, 2024. URL https://en.wikipedia.org/wiki/Domain-specific_language. Accessed on 2024 May 11.

Appendix A

Appendix: Erlang Build Errors

Full log URL:

https://github.com/iatyl/goldsmiths-final-project/blob/main/logs/kerl_errors.log

Appendix B

Appendix: Create User Accounts in iex REPL

```
Interactive Elixir (1.15.7) - press Ctrl+C to exit (type h() ENTER for
↪ help)
iex(1)> alias Irchub.Repo
Irchub.Repo
iex(2)> alias Irchub.Accounts.User
Irchub.Accounts.User
iex(3)> user = %User{email: "hi@live.com"}
#Irchub.Accounts.User<
  __meta__: #Ecto.Schema.Metadata<:built, "users">,
  id: nil,
  email: "hi@live.com",
  confirmed_at: nil,
  inserted_at: nil,
  updated_at: nil,
  ...
>
```

```
iex(4)> changeset = User.password_changeset(user, %{password:
↪  "Ies6vei2aeth7sa"})
#Ecto.Changeset<
  action: nil,
  changes: %{hashed_password: "**redacted**"},
  errors: [],
  data: #Irchub.Accounts.User<>,
  valid?: true
>
iex(5)> Repo.insert changeset
[debug] QUERY OK source="users" db=15.6ms decode=2.6ms queue=1.0ms
↪  idle=1967.8ms
INSERT INTO "users" ("email","hashed_password","inserted_at","updated_at")
↪  VALUES ($1,$2,$3,$4) RETURNING "id" ["hi@live.com",
↪  "$2b$12$D5EpyggObwsWIyxvCrHsL.AuSDxBMPWEl0Aw/7dPWGL0qwghJ1dOy",
↪  ~U[2024-03-24 21:32:35Z], ~U[2024-03-24 21:32:35Z]]
 :elixir.eval_external_handler/3, at: elixir.erl:396
{:ok,
 #Irchub.Accounts.User<
   __meta__: #Ecto.Schema.Metadata<:loaded, "users">,
   id: 6,
   email: "hi@live.com",
   confirmed_at: nil,
   inserted_at: ~U[2024-03-24 21:32:35Z],
   updated_at: ~U[2024-03-24 21:32:35Z],
   ...
 >}
iex(6)>
```

Appendix C

Appendix: Database Migration Output

```
The database for Irchub.Repo has been created
[info] == Running 20240204181615
↪  Irchub.Repo.Migrations.CreateUsersAuthTables.change/0 forward

[info] execute "CREATE EXTENSION IF NOT EXISTS citext"

[info] create table users

[info] create index users_email_index

[info] create table users_tokens

[info] create index users_tokens_user_id_index

[info] create index users_tokens_context_token_index

[info] == Migrated 20240204181615 in 0.0s
```

```
[info] == Running 20240204231455
↪ Irchub.Repo.Migrations.CreateClients.change/0 forward

[info] create table clients

[info] == Migrated 20240204231455 in 0.0s
```

Appendix D

Appendix: ExIRC Connect to Server

```
# spawn a new IRC client
{:ok, client} = ExIRC.start_client!

# connect the client to irc://localhost:6667
ExIRC.Client.connect! client, "localhost", 6667

# authentication
ExIRC.Client.logon client, "password", "nick", "user", "fullname"

# join #chat
ExIRC.Client.join client, "#chat"

# say "Hello world!" in #chat
ExIRC.Client.msg client, :privmsg, "#chat", "Hello world!"

# disconnect from server with a message "client quit"
ExIRC.Client.quit client, "client quit"

# stop the client
ExIRC.Client.stop! client
```

Appendix E

Appendix: *Mix test* Results

Full log is available at the following URL:

https://github.com/iatyl/goldsmiths-final-project/blob/main/logs/tmp_mix_test.txt